EasiGP

User Manual

Ensemble AnalySis with Interpretable Genomic Prediction
A step-by-step guide for new and experienced users

October 2026 | https://github.com/ShunichiroT/EasiGP

Contents

1. About EasiGP 5

2. Genomic prediction in plain words 6

2.1 The basic idea 6

2.2 Training, validation and test sets 6

2.3 Measuring accuracy 7

2.4 Why run several models? 7

2.5 Ensembles 8

2.6 Interpreting models 8

3. Installing and launching 9

3.1 Route A step by step (laptop with Docker) 9

3.2 Route A on an HPC cluster (Apptainer) 10

3.3 Route B step by step (conda) 11

3.4 Starting the GUI 11

3.5 The interface at a glance 12

4. Preparing your input data 13

4.1 What the files look like 13

4.2 Rules that avoid most errors 14

5. Choosing an execution mode 15

6. Your first run 16

6.1 Quick start with the example data 16

6.2 The configuration flow 17

6.3 Using your own data: a checklist 17

7. Tab 1 – Data & Setup 18

8. Tab 2 – Data Preprocessing (optional) 21

8.1 LD pruning 21

8.2 RF marker importance filtering 22

8.3 ASReml-R mixed-model adjustment (BLUE/BLUP) 23

8.4 Biological prior network 24

9. Tab 3 – Models & Hyperparameters 26

9.1 Available models 26

9.2 The models in plain words . . . 28
9.3 Fields common to most model panels . . . 29

10. Tab 4 – Ensemble . . . 32

11. Tab 5 – Violin, Bar & Scatter Plots . . . 34

12. Tab 6 – Circos Plot . . . 36

13. Tab 7 – Run pipeline . . . 39

13.1 Option A fields . . . 40
13.2 Parallel mode in practice . . . 41

14. Tool mode – comparing finished runs . . . 42

15. Output files . . . 43

16. Reading your results . . . 44

16.1 Accuracy violin plots . . . 44
16.2 Metric_summary.xlsx . . . 44
16.3 Ensemble weights (Weight.png, Weight.csv) . . . 44
16.4 Diversity Prediction Theorem . . . 44
16.5 Ranking-based metrics . . . 45
16.6 Scatter-plot matrix (model pairwise comparison) . . . 45
16.7 Circos plots . . . 45

17. Recommended settings by goal . . . 47

17.1 Managing run time . . . 47

18. Troubleshooting and FAQ . . . 48

18.1 Finding the real error message . . . 48
18.2 General problems . . . 48
18.3 Docker and Apptainer . . . 49
18.4 R and rpy2 . . . 53
18.5 Python and conda environment . . . 56
18.6 HPC jobs and job scripts . . . 56
18.7 Frequently asked questions . . . 57

Appendix A. Glossary . . . 59

Appendix B. Configuration checklist . . . 61

Appendix C. Hyperparameter reference 62

C.1 Which setting should I change? 63

C.2 Explanation methods in plain words 63

C.3 rrBLUP, BayesB and GBLUP 64

C.4 RKHS 66

C.5 Tree models: RF, ExtraTrees and XGBoost 68

C.6 EBM 72

C.7 SVR 72

C.8 KNN 74

C.9 MLP (neural network) 75

C.10 Graph attention (GAT) models 77

C.11 Weight-optimisation ensemble methods (Tab 4) 79

1. About EasiGP

EasiGP predicts a trait (for example yield, flowering time or disease resistance) from DNA markers. It runs several different prediction models on the same data, combines them into **ensembles**, and shows what each model has learned about the genome as a **circos plot**. Everything is set up in a web-based graphical interface (GUI) that opens in your browser; no programming is needed for day-to-day use.

What you can do with it

- **Predict traits** with up to 15 individual models (BLUP/Bayesian, tree-based, kernel, neural-network and graph attention models).
- **Combine models** into a naive (equal-weight) ensemble and/or weighted ensembles.
- **Interpret models**: marker effects, marker-pair interactions and attention weights, drawn side by side on circos plots together with known gene regions.
- **Prepare data** (optional): LD pruning, Random-Forest marker filtering, ASReml-R BLUE/BLUP adjustment, and a gene-interaction network for the biological-prior GAT model.
- **Scale up**: run on a laptop or generate HPC job scripts (Slurm/PBS), including job arrays that resume automatically after a failure.
- **Compare finished runs** in Tool mode without refitting anything.

What a run gives you

Output	What it tells you
Accuracy tables and violin plots	How well each model (and each ensemble) predicted individuals it had never seen.
Excel summary reports	Means, medians and standard errors per model, trait and population, ready for a paper or report.
Marker effects	Which DNA markers each model relied on most.
Marker-pair interactions	Which pairs of markers act together (epistasis-like signals).
Circos plots	All of the above drawn on one genome map, next to genes already known for the trait.

How to use this manual

If you are...	Read
New to genomic prediction	Section 2 (concepts), then Section 6 (first run), then Section 16 (reading results). Keep Appendix A (glossary) at hand.
Experienced, new to EasiGP	Sections 3–6, then the tab reference (Sections 7–13) as needed. Appendix C explains every model setting.
Running on an HPC cluster	Sections 5, 13 and 15, then Section 17 for recommended settings.
Stuck on an error	Section 18 (troubleshooting and FAQ).

> **Tip**
> Every field in the GUI has a **?** icon. Hover over it for a detailed explanation. This manual explains *why* you would change a field; the ? icon gives the exact technical detail.

2. Genomic prediction in plain words

This section explains the ideas behind EasiGP for readers without a background in genomic prediction or machine learning. If you already know the field, skip to Section 3.

2.1 The basic idea

Many traits of plants and animals are partly controlled by their DNA. A **DNA marker** (usually a SNP, single-nucleotide polymorphism) is a position in the genome where individuals differ. For each individual and marker we record how many copies of one particular variant (allele) it carries: **0**, **1** or **2**. The table of these numbers is the **genotype**. The measured trait values (e.g. yield in t/ha) are the **phenotype**.

Genomic prediction uses individuals that have both genotype and phenotype to teach a model how markers relate to the trait. The model can then predict the trait of new individuals from their DNA alone, for example seedlings that have not yet been grown in the field. Breeders use these predictions (often called genomic estimated breeding values, GEBVs) to choose parents and lines earlier and more cheaply.

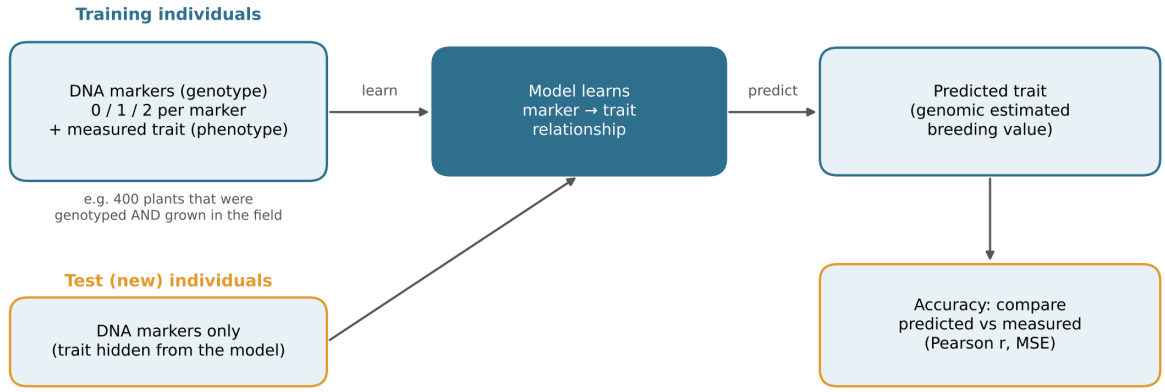


Figure 1. Genomic prediction in one picture. The model learns from training individuals and is checked on test individuals whose trait values it never saw.

2.2 Training, validation and test sets

To find out how good a model really is, EasiGP hides part of the data from it:

- **Training set**: individuals the model learns from.
- **Validation set** (optional): individuals used to make choices *about* the model, such as tuning its settings or deciding how much weight each model gets in an ensemble.
- **Test set**: individuals kept completely aside until the end. Accuracy is always measured here, so the result tells you how well the model would do on genuinely new material.

Because the result depends on which individuals happen to land in each set, EasiGP repeats the random split several times (**replicates**, also called iterations) and reports the spread. The split is written as a **split ratio**: `0.8` means 80 % training and 20 % test; `(0.8, 0.1, 0.1)` means 80 % training, 10 % validation and 10 % test.

> **Why the test set must stay hidden**
>
> If any information from the test individuals is used during training (for example, to choose which markers to keep), the accuracy looks better than it really is. This is called *data leakage*. EasiGP prevents it automatically: marker filtering, tuning and ensemble weights only ever use the training and validation sets.

2.3 Measuring accuracy

Measure	What it means	Better when
Pearson correlation (r)	How closely predicted values follow measured values. Ranges from −1 to 1. This is the most widely reported "prediction accuracy".	Higher (closer to 1)
MSE (mean squared error)	The average squared difference between predicted and measured values, in the trait's own units squared. Only compare MSE between models on the same trait.	Lower (closer to 0)
Spearman correlation	Like Pearson, but based on ranks. Useful when you care about ordering individuals.	Higher
Top-k Coincidence	Of the truly best k % of individuals, what share does the model also put in its top k %? Close to what a breeder does when selecting.	Higher
Heritability (h^2)	The share of trait variation that is due to genetics (0 to 1). It limits how accurate any prediction can be (only with the ASReml-R option).	Not a model score



Figure 2. What different Pearson correlations look like. Each dot is one test individual.

What is a "good" accuracy? It depends mostly on the trait. Highly heritable traits such as flowering time are often predicted with r of 0.6–0.9; complex, low-heritability traits such as yield often give 0.2–0.5. As a rule of thumb, r rarely exceeds the square root of the heritability. Judge models *against each other on the same data* rather than against a fixed target.

2.4 Why run several models?

Different models make different assumptions about how genes act. Some assume many genes with tiny additive effects (rrBLUP, GBLUP); some assume a few genes with large effects (BayesB); others can capture genes that act together (Random Forest, neural networks, graph attention models). The truth for a given trait is usually unknown, so no single model wins everywhere. Running several models side by side shows which assumptions suit your trait, and Section 9.1 helps you choose.

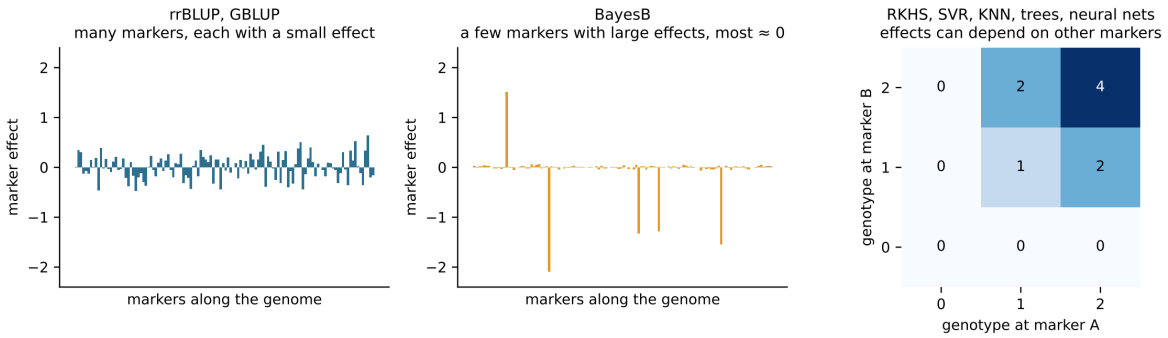


Figure 3. What the main model families assume. Left and middle: the marker effects each linear model expects along the genome. Right: an interaction, where the effect of marker A depends on marker B (numbers are trait values); only non-linear models can learn this.

2.5 Ensembles

An **ensemble** combines the predictions of several models, for example by averaging them. Errors of different models partly cancel out, so an ensemble is often as good as or better than the best single model and is more reliable across traits. The **naive ensemble** gives every model equal weight. **Weighted ensembles** learn on the validation set how much to trust each model. The **Diversity Prediction Theorem** (Section 16.4) explains why: ensemble error = average model error − how much the models disagree. Diverse, reasonably accurate models make the best ensembles.

2.6 Interpreting models

EasiGP also asks each model *why* it predicts what it does:

- A **marker effect** says how strongly a marker influences the model's predictions. For linear models this is the model coefficient; for other models it is estimated with **Shapley values** (SHAP), a standard method that shares a prediction fairly among the markers.
- An **interaction** score says how strongly two markers act *together*, beyond their separate effects.
- **Attention weights** (graph attention models only) show which marker connections the model paid most attention to.

These are drawn on a circos plot (Section 12) so you can see whether different models point to the same genome regions, and whether those regions contain genes already known to affect the trait. Regions that several models agree on, but that contain no known gene, are candidates for new discoveries.

> **Keep in mind**
>
> Marker effects show what a *model* relied on, not proof that a gene causes the trait. Markers close to each other on a chromosome are often inherited together (linkage disequilibrium), so a high-effect marker may simply be near the causal gene.

3. Installing and launching

There are two ways to install EasiGP. This section gives the commands from the README of the GitHub repository (https://github.com/ShunichiroT/EasiGP), which also has extra detail for HPC clusters.

Route	Where	What you do	Choose it when
A. Ready-made image (recommended)	Laptop: Docker HPC: Apptainer	Pull `shunichirot/easigp` from Docker Hub and run it; nothing else to install.	You want the simplest setup.
B. From source code	Laptop or HPC (conda)	Clone the repository, create the conda environment from `Environments/*.yml`, install `plink2` if you need LD pruning.	You want to read/modify the code, or use ASReml-R (not included in the images).

Full vs Light version. Both contain all models, tuning, preprocessing and plots. The Full version adds the Claude Code integration used to generate gene-interaction networks with FLASH-P automatically (needs a Claude access token). With the Light version you can still supply a network file yourself. If you are unsure, choose Light.

> **New to Docker? Three words**
>
> An **image** is a ready-made package of EasiGP with Python, R and every library already installed. A **container** is a running copy of that image. **Docker** runs containers on a laptop; **Apptainer** (formerly Singularity) does the same on HPC clusters, where Docker is usually not allowed. Nothing you do inside a container is kept after it stops, unless it is written to a folder you **mounted** from your own computer.

3.1 Route A step by step (laptop with Docker)

1. Install **Docker Desktop** (Windows/macOS) or Docker Engine (Linux), start it, and wait until it reports that it is running. On Windows, Docker Desktop needs WSL 2 and hardware virtualisation (Section 18.3 if it does not start).
2. Look up the current tag on the Docker Hub page (https://hub.docker.com/r/shunichirot/easigp/tags): `<version>` for the Full version, `<version>_light` for the Light version.
3. Open a terminal (Windows: PowerShell; macOS/Linux: Terminal) and download the image once (several GB): `docker pull shunichirot/easigp:<version>_light`
4. Create two folders on your computer, e.g. *EasiGP_work/data* for your input files and *EasiGP_work/Result* for the results, and change into *EasiGP_work* in the terminal.
5. Start EasiGP with the command below for your system. Leave the terminal open while you work.
6. Open `http://localhost:8501` in your browser (not the “Network URL” printed in the terminal). In Tab 1, give your input files as `/workspace/data/<file>.csv`.
7. When finished, press **Ctrl+C** in the terminal. Your results are in *EasiGP_work/Result*.

macOS / Linux (Terminal), Light version:

```
docker run -it -p 8501:8501 --shm-size=8g \
  -v "$(pwd)/data":/workspace/data \
  -v "$(pwd)/Result":/workspace/EasiGP/Result \
  shunichirot/easigp:<version>_light \
  streamlit run main_app.py --server.address=0.0.0.0 --server.port=8501
```

Windows (PowerShell), Light version (each line ends with a backtick, PowerShell's line continuation):

```
docker run -it -p 8501:8501 --shm-size=8g `
  -v "${PWD}\data:/workspace/data" `
  -v "${PWD}\Result:/workspace/EasiGP/Result" `
  shunichirot/easigp:<version>_light `
  streamlit run main_app.py --server.address=0.0.0.0 --server.port=8501
```

For the **Full** version, use the tag `<version>` and add `-e CLAUDE_CODE_OAUTH_TOKEN=<your token>` before the image name (create the token once with `claude setup-token` on a computer with a browser; keep it private). To use an NVIDIA GPU, add `--gpus all` (needs the NVIDIA driver and, on Linux, the NVIDIA Container Toolkit).

Part of the command	Why it is there
`-p 8501:8501`	Connects port 8501 of your computer to the GUI inside the container. If 8501 is busy, use e.g. `-p 8502:8501` and open http://localhost:8502.
`--shm-size=8g`	Gives the container more shared memory. Docker's default (64 MB) is too small for EasiGP's parallel workers and PyTorch (Section 18.3).
`-v "...data":/workspace/data`	Makes your data folder visible inside the container as /workspace/data.
`-v "...Result":/workspace/EasiGP/Result`	Saves every result folder on your own computer. **Without it, results are lost when the container is removed.**
`--server.address=0.0.0.0`	Lets the browser on your computer reach the GUI inside the container. Without it the page does not load.

Things that are not kept between Docker sessions

Each `docker run` starts a fresh container. Only mounted folders survive: your remembered GUI settings (easigp_last_gui_state.json) are not kept, so save important configurations by exporting the job files (Tab 7, Option A) into the mounted Result folder, and reload them with **Load a previous configuration** (Tab 1). Stopped containers still use disk space; remove them with `docker container prune`.

3.2 Route A on an HPC cluster (Apptainer)

1. Log in and check Apptainer: `apptainer --version` (or `module load apptainer`).
2. Build a sandbox (a folder copy of the image) once, in your scratch space: `apptainer build --sandbox /scratch/user/$USER/EasiGP docker://shunichirot/easigp:<version>`. If the login node has no internet, run `docker save -o EasiGP.tar shunichirot/easigp:<version>` on a computer with Docker, copy the file over, and build from `docker-archive:///path/to/EasiGP.tar` instead.
3. Start an interactive job on a compute node (e.g. `salloc` on Slurm; never run EasiGP on the login node).
4. Run `apptainer run /scratch/user/$USER/EasiGP`, then `cd /scratch/user/$USER/EasiGP/workspace/EasiGP/` and `streamlit run main_app.py`

`--server.address=0.0.0.0 --server.port=8501`. For the Full version, first `export APPTAINERENV_CLAUDE_CODE_OAUTH_TOKEN="<your token>"`.

5. On your own computer, open an SSH tunnel to that compute node: `ssh -N -L 8501:<node>:8501 <user>@<hpc login address>` and open http://localhost:8501. On UQ Bunya (onBunya) or NCI Gadi (ARE), a virtual desktop replaces the interactive job and the tunnel.

3.3 Route B step by step (conda)

1. Install Miniconda or Anaconda. On an HPC, load it (e.g. `module load anaconda3`) and do the installation on the login node, which usually has internet access.
2. Clone the repository: `git clone https://github.com/ShunichiroT/EasiGP`, then `cd` into the main EasiGP folder.
3. Create the environment (this downloads many packages): Windows (Anaconda Prompt) `conda env create -f Environments/environment_windows.yml`; Linux `conda env create -f Environments/environment_linux.yml`. The files pin exact versions of Python 3.11, R, BGLR and rpy2 that are known to work together.
4. Activate it: `conda activate EasiGP`. Do this every time before starting EasiGP.
5. Optional: install `plink2` and put it on your PATH (LD pruning, PLINK input). XGBoost and EBM are already pinned in the environment files.
6. Start the GUI (Section 3.4). To update later, `git pull` and, if the package list changed, `conda env update -f Environments/environment_<os>.yml --prune`.

> **Optional: ASReml-R (paid licence, Route B only)**
> ASReml-R is not on conda, CRAN or pip and is not in the Docker images. With the EasiGP environment **activated**, start R and run `install.packages("<downloaded asreml file>", repos = NULL)` (add `type = "win.binary"` for the Windows .zip, `type = "source"` for the Linux .tar.gz). The package must match the R version of the environment (check with `R --version`). Activate the licence from Tab 2 or with `asreml::asreml.license.activate()`. Activation is per machine: on an HPC, activate it where the jobs run.

3.4 Starting the GUI

With Docker or Apptainer the GUI starts with the command of Section 3.1 or 3.2. With conda, from the main EasiGP folder (environment activated; on an HPC inside an interactive job, adding `--server.headless true`):

```
conda activate EasiGP
streamlit run main_app.py --server.port 8501
```

Then open `http://localhost:8501` in a browser (it usually opens by itself). On an HPC, open an SSH tunnel from your own computer as in Section 3.2, or use a virtual desktop (e.g. UQ Bunya onDemand, NCI ARE). Press **Ctrl+C** in the terminal to stop.

> **Do not close the terminal**
> The terminal window runs EasiGP. Closing it stops the GUI and any run started with **Option B: run now**. Progress messages and errors are also printed there, so it is the first place to look if something goes wrong.

3.5 The interface at a glance

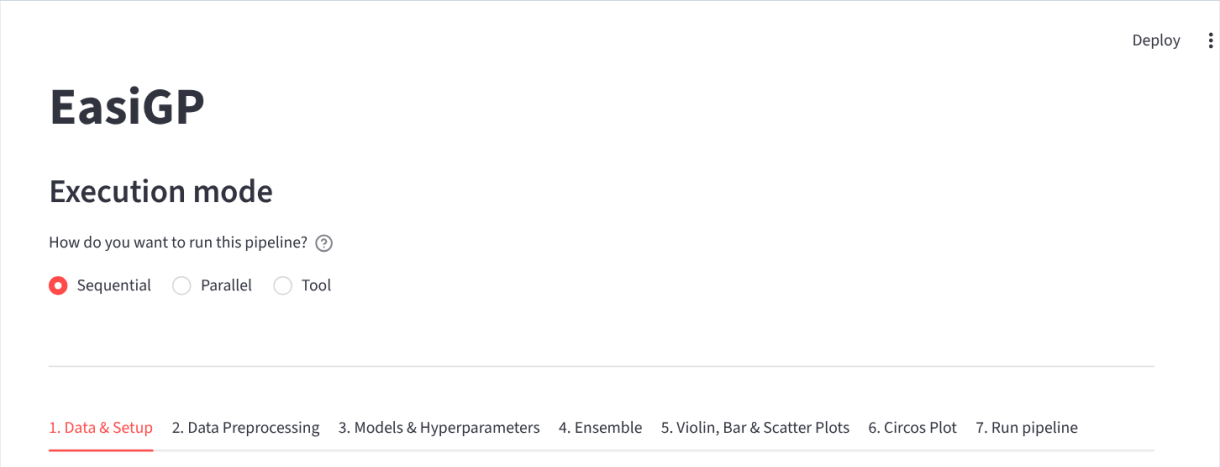


Figure 4. Top of the GUI: the execution mode (Sequential, Parallel or Tool) and the configuration tabs.

- **Execution mode** at the top decides which tabs appear (Section 5).
- **Tabs 1–7** are worked through from left to right (Section 6.2).
- Every field has a **?** icon: hover over it for a detailed explanation of that field.
- Many plot fields show **auto** or are pre-filled from your data; leave them as they are unless you want to override them.
- Green messages (e.g. “Found”) confirm a setting is valid; yellow and red messages explain what still needs attention.
- **Your settings are remembered**: the GUI saves its state automatically and restores it the next time it starts. Use **Load a previous configuration** (top of Tab 1) to restore the settings of an earlier run from its saved configuration file.

4. Preparing your input data

EasiGP reads plain CSV files (comma-separated text files that you can create in Excel with *Save As → CSV*). Only the genotype and phenotype files are needed to fit models; the others are needed for circos plots or for specific add-ons.

File	Needed?	Columns (in this order)
Genotype CSV	Yes (or PLINK)	ID, population, then one column per marker. Values are 0/1/2 allele counts.
Genotype PLINK1 fileset	Alternative to CSV	`stem.bed / .bim / .fam` sharing one file stem; useful for large marker sets.
Phenotype CSV	Yes	ID, population, then one column per trait. In “Raw” layout it may also hold design columns (block, row, year, ...).
marker_info.csv	For circos plots	chromosome, name, start, end. *name* must match the genotype marker columns exactly.
chrom.csv	For circos plots	chromosome, start (usually 0), end (length), optional population.
gene_info.csv	Optional	chromosome, start, end, name, colour (category), source (ring label), phenotype, optional population.
network.json + gene location table	Bio-prior GAT only	Which genes interact, and where each gene is (Gene_Name, Chromosome, Start_bp/End_bp or Start_cM/End_cM).

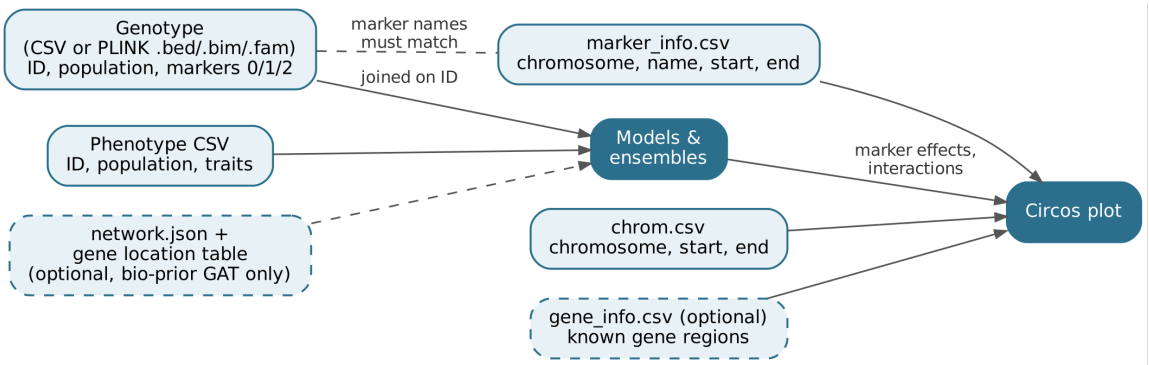


Figure 5. How the input files connect. Dashed boxes are optional.

4.1 What the files look like

The first rows of each file, using made-up values. Your column names may differ, but the order must be the same.

Genotype CSV (one row per individual, one column per marker)

ID	population	SNP_001	SNP_002	SNP_003	...
Z001E0001	pop1	0	2	1	...
Z001E0002	pop1	2	2	0	...
Z002E0001	pop2	1	0	0	...

Phenotype CSV, “Analysed” layout (one row per individual, one column per trait)

ID	population	days2anthesis	plant_height	yield
Z001E0001	pop1	71.2	182.0	6.41
Z001E0002	pop1	68.9		5.97
Z002E0001	pop2	74.5	190.3	7.02

marker_info.csv and chrom.csv (circos plots)

chromosome	name	start	end
1	SNP_001	0.0	0.0
1	SNP_002	1.6	1.6
2	SNP_003	0.4	0.4

chromosome	start	end
1	0	160.4
2	0	135.8

gene_info.csv (optional known genes)

chromosome	start	end	name	colour	source	phenotype
1	45.1	45.3	GeneA	flowering	QTL study X	days2anthesis
8	70.2	70.9	GeneB	flowering	QTL study X	days2anthesis

Positions in `marker_info.csv`, `chrom.csv` and `gene_info.csv` must use the **same unit**: either base pairs (bp) or centiMorgans (cM). For a single marker, start and end are the same value. The *phenotype* column of `gene_info.csv` must match a trait name exactly, so each gene is drawn only on the circos plots of that trait.

4.2 Rules that avoid most errors

- The **first two columns** of the genotype and phenotype files are read **by position** (ID, population), whatever their header says. All other columns are matched **by name**.
- Marker, chromosome and gene-info files are also read by column position: keep the column order shown above.
- Individuals are matched between the genotype and phenotype files by **ID**. Individuals present in only one file are left out.
- Each ID should appear only once. If an ID is repeated (e.g. the same line grown in two environments), EasiGP keeps the first row and warns you. Summarise repeated measurements first, or use the “Raw” layout with ASReml-R (Section 8.3).
- **Missing trait values** (empty cells) are fine: an individual is simply left out of the analysis of that trait.
- **Missing genotype values**: in a genotype CSV, an individual with any missing marker value is dropped, so impute missing genotypes beforehand (e.g. with Beagle) or remove markers with many missing values. With PLINK input, missing calls are filled with the marker’s average.
- Chromosome names must be identical across `chrom.csv`, `marker_info.csv` and `gene_info.csv` (e.g. *10* everywhere, not *10* and *A10*).
- With PLINK input, ID and population still come from the phenotype file; the .fam IDs are only used for matching.
- If you only have one population, put the same label (e.g. *pop1*) in every row of the population column.

- Avoid spaces and special characters in trait and marker names; use letters, digits and underscores.

> **Example data**
> EasiGP ships with `Data/MaizeNAM`, `Data/TeoNAM` and `Data/Arabidopsis`. The GUI's default paths point to MaizeNAM population 1, so you can try EasiGP straight away. Unzip any `.zip` files first. Opening these files in Excel is the quickest way to see a correct layout.

5. Choosing an execution mode

The execution mode, chosen at the very top of the GUI, decides *how* the work is run. A run is made of many **tasks**: one task is one population (or population pair) × one trait × one split ratio × one replicate. For example, 2 traits × 3 populations × 10 replicates = 60 tasks, and every selected model is fitted in each task.

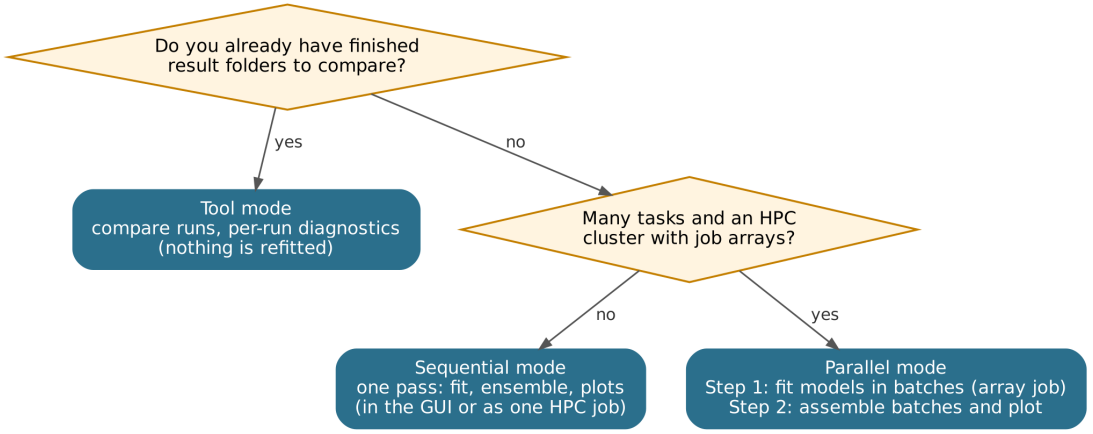


Figure 6. Which execution mode to choose.

Mode	What it does	Tabs shown
Sequential	Runs everything in one pass: data setup, model fitting, ensembles, plots. Run it in the GUI or export it as a single HPC job.	1, 2, 3, 4, 5, 6, 7
Parallel – Step 1	Fits the models, split into batches of tasks that run as independent HPC array sub-jobs.	1, 2, 3, 4, 7
Parallel – Step 2	Assembles the results of all batches and produces every plot and report.	1, 4, 5, 6, 7
Tool	Compares finished runs and runs per-run diagnostics. Nothing is refitted.	Its own two tabs

Rough guide

Situation	Mode
First try, learning EasiGP, small data (up to a few thousand markers, tens of tasks)	Sequential, run in the GUI (Option B)
Medium analysis that fits in one long job	Sequential, exported as one HPC job (Option A)
Hundreds of tasks, slow models (neural networks, GAT, tuning) or many markers	Parallel Step 1 then Step 2 on an HPC

Situation	Mode
You already have finished result folders and want to compare them	Tool

> **Tip**
> Start with Sequential mode on a small dataset (e.g. the MaizeNAM example) to check that your configuration works, then switch to Parallel mode for the full analysis on an HPC. Settings are kept when you switch mode.

6. Your first run

6.1 Quick start with the example data

This run uses only default values and takes a few minutes on a laptop. It fits three models to maize flowering time and draws all plots, so you can see every kind of output before using your own data.

1. Start the GUI (Section 3.4) and select **Sequential** as the execution mode.
2. **Tab 1**: keep the MaizeNAM population 1 paths (check that both show a green “Found” message), choose a **Target phenotype** such as *days2anthesis* (days to flowering), and set **Number of random-split iterations** to **5** (the default is 1, and a violin plot needs at least 5 replicates).
3. **Tab 2**: leave everything off.
4. **Tab 3**: keep the models ticked by default (rrBLUP, BayesB, RF).
5. **Tab 4**: keep **Naive ensemble (equal weight)** ticked.
6. **Tab 5**: keep the defaults.
7. **Tab 6**: keep the default chromosome, marker and gene info paths.
8. **Tab 7**: choose **Option B: run now (local)** and click **Run pipeline now (local)**. Progress appears in the GUI and in the terminal.
9. When it finishes, open the result folder (`Result/<Result folder name>/`) and look at `Pearson correlation.png`, `Metric_summary.xlsx` and the `circos_*.png` files.

What you should see

- **Pearson correlation.png**: one violin per model and one for the ensemble. Each violin shows the accuracy over all replicates; a narrow violin means a stable result. Flowering time is highly heritable, so expect fairly high r.
- **Metric_summary.xlsx**: the same numbers as a table (mean, median and standard error per model).
- **circos_days2anthesis_*.png**: one ring of marker effects per model plus the ensemble. Look for dark regions that appear in several rings at the same position, and whether they line up with the known flowering genes.

Section 16 explains each of these outputs in detail. Next, try ticking more models in Tab 3 or increasing the number of replicates in Tab 1, and run again under a new result folder name to compare.

6.2 The configuration flow

Work through the tabs from left to right. Each tab feeds the next, and Tab 7 finally runs the pipeline (or writes an HPC job script).

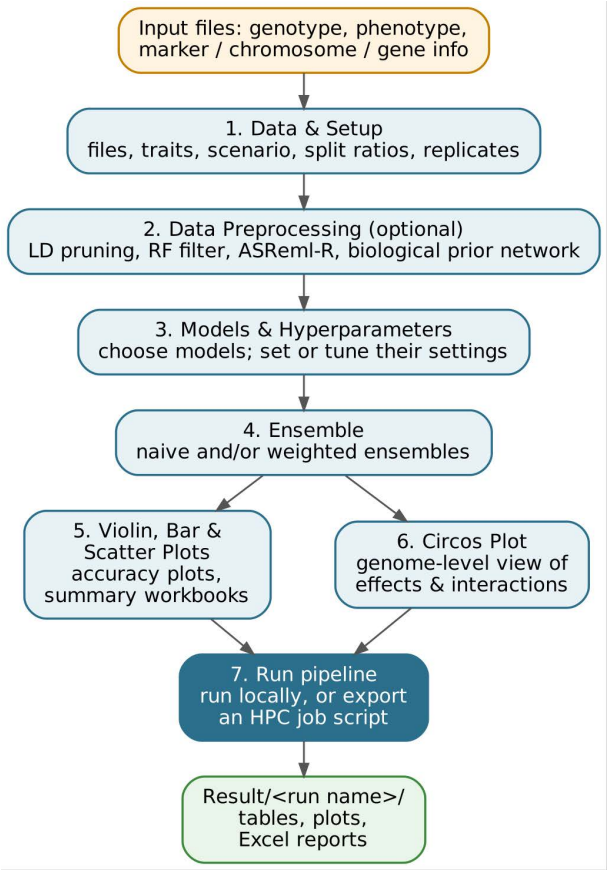


Figure 7. Configuration flow through the tabs (Sequential mode).

6.3 Using your own data: a checklist

1. Prepare the genotype and phenotype files (Section 4). Open them once in Excel to check the first two columns are ID and population.
2. Run one trait, one population and 2–3 replicates with fast models (rrBLUP, RF) to check that everything works.
3. Add the circos files (`marker_info.csv`, `chrom.csv`, optionally `gene_info.csv`) and check the circos plot.
4. Choose the models and ensembles you really want (Section 17) and increase replicates (10 or more).
5. For large analyses, export the configuration and run it on an HPC (Section 13).

7. Tab 1 – Data & Setup

In short: tells EasiGP where the data are, which traits to predict, how individuals are split into training/validation/test sets, and where to save the results.

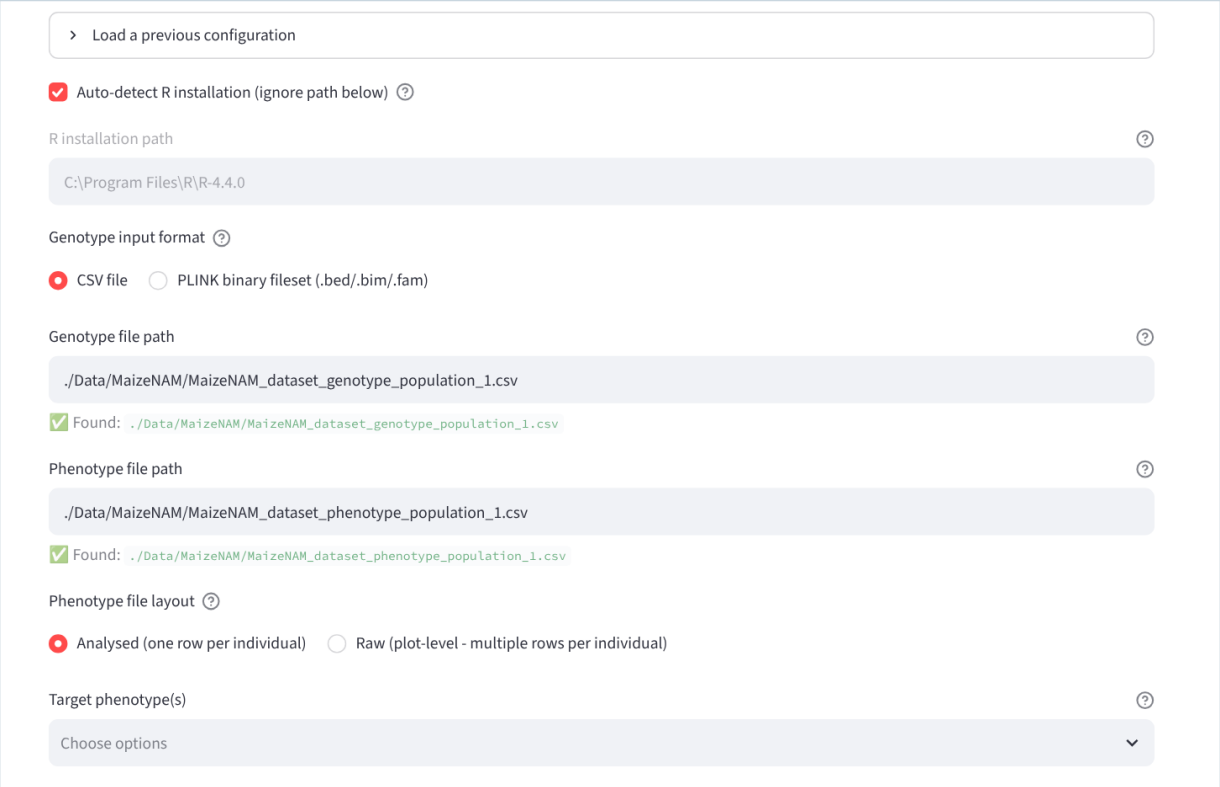


Figure 8. Top of Tab 1 with its default values.

Field	Role
Load a previous configuration	Restores every field from a saved configuration file of an earlier run (pick from the list, type a path, or upload). Useful to repeat or slightly change an earlier analysis.
Auto-detect R installation (ignore path below) / R installation path	R runs rrBLUP, GBLUP, BayesB and RKHS. Keep auto-detect ticked (default): rpy2 then uses the R of the active EasiGP environment. Untick only if R is not found, and enter the folder of the R that has BGLR installed. The pre-filled `C:\Program Files\R\R-4.4.0` is only an example and is ignored while auto-detect is ticked (Section 18.4).
Genotype input format	**CSV file** (default) or **PLINK binary fileset (.bed/.bim/.fam)**. For PLINK, enter the **PLINK file stem** (the path without .bed/.bim/.fam) and the **PLINK2 executable** (default `plink2`). EasiGP converts only the markers each step needs, as late as possible.
Genotype / Phenotype file path	Paths to your files, relative to the EasiGP folder (e.g. `./Data/...`) or absolute. A green “Found” message confirms the file exists.
Phenotype file layout	**Analysed (one row per individual)** (default) or **Raw (plot-level - multiple rows per individual)**, needed for the ASReml-R stage. In Raw layout three more fields appear: **Trait column(s) in this file**; **Population source** = *Derived from several column(s)* (default; e.g. Location then Year gives “Roseworthy_2014”) or *One constant label*; and either **Column(s) to derive population from** or **Population label** (default `all_lines`).

Field	Role
Target phenotype(s)	Trait column(s) to predict: pick them from the list (or type them separated by commas, or *all*, if the file cannot be read yet). Each trait is analysed separately.
Phenotype direction overrides (optional) / Direction for <trait>	Used by Top-k Coincidence and Czekanowski. By default the “top” individuals are those with the highest values (e.g. yield). Pick the traits where lower is better (e.g. disease score, days to maturity) and set their direction.
Result folder name	Results are written to `Result/<name>/`. Use a new, descriptive name for each analysis (e.g. `maize_DTA_5models`). Re-using a name resumes an unfinished run.
Result file compression	*gzip* (default) compresses the large result tables (.csv.gz); *none* writes plain CSV. Most spreadsheet programs cannot open .gz directly; unzip them first or choose *none* for small runs.
Prediction scenario	**within**: train and test within each population. **between**: train on one population, test on another (Figure 9).
Split ratio(s)	within (default `0.8`): a training share such as *0.8*, or a (train, validation, test) tuple such as *(0.8, 0.1, 0.1)*. Several values separated by commas are each run separately. between: only needed when the validation set is split from a population (default `(0.8,0.1,0.1)`).
Number of random-split iterations	How many replicates of the random split are run (default **1**). Each replicate uses a different, reproducible random split. Raise it (5 or more) to get meaningful violin plots and standard errors.
Use a validation set / Validation set source	between only. Sources: *third_population* (default; train, validation and test each come from a different population), *split_train* (part of the training population) or *split_test* (part of the test population).
Population combination(s) to exclude	between only: remove specific train→test (or train→validation→test) combinations.
Minimum data points required per iteration	Tasks with fewer individuals than this (after removing missing values) are skipped. Default 100.

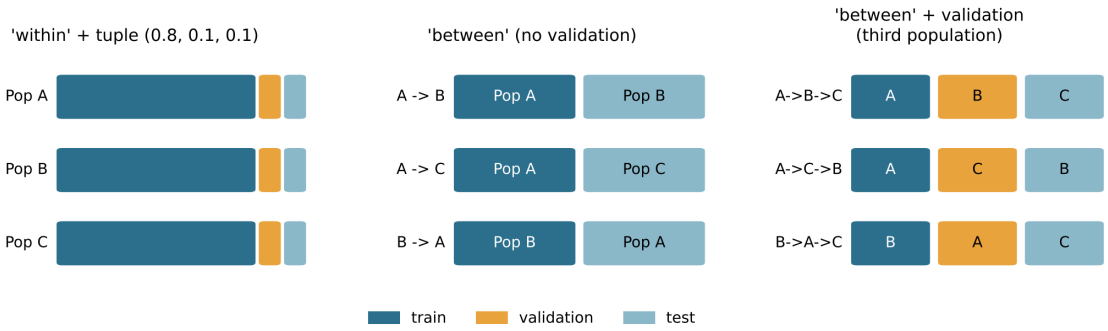


Figure 9. How individuals are assigned in the within and between scenarios.

Choosing a scenario

- **within** answers “how well can I predict new individuals from the same population?” This is the usual choice and the right one if you have only one population.
- **between** answers “can a model trained on one population predict a different population?” This is harder, and accuracy is usually lower, especially when the populations are distantly related.

How many replicates?

More replicates give a more reliable average and show how stable each model is, but the run time grows in proportion. Use 2–3 replicates for testing, about 10 for a normal analysis and 20 or more when you need to show that small differences between models are real.

> **Important: when you need a validation split**
>
> Any **weight-optimisation ensemble** (Tab 4) and any **hyperparameter tuning** (Tab 3) need a validation set. In the *within* scenario, type the split ratio as a tuple, e.g. `(0.8, 0.1, 0.1)`; in the *between* scenario, tick **Use a validation set**. Otherwise a plain number such as `0.8` is enough.

8. Tab 2 – Data Preprocessing (optional)

In short: optional steps that clean or reduce the data before modelling. Beginners can leave this tab untouched. Four independent sections; all are off by default. LD pruning and RF filtering are decided from each task's **training set only**, so no information leaks from the test set. When both are on, LD pruning runs first and RF filtering narrows the remaining markers.

Section	Use it when
LD pruning	You have many closely spaced markers (e.g. tens of thousands of SNPs) and want to remove redundant ones to save time.
RF marker filtering	Slow models (neural networks, GAT, SVR) take too long and you want to keep only the most informative markers.
ASReml-R adjustment	Your phenotypes are raw field-plot records (replicates, blocks, years) and you have an ASReml-R licence.
Biological prior network	You use the GAT_biological_prior_knowledge model.

8.1 LD pruning

Markers that lie close together are often inherited together, so they carry almost the same information. This is called **linkage disequilibrium (LD)** and is measured with r^2 (0 = independent, 1 = identical). LD pruning uses plink2 to remove one marker of every highly correlated pair, giving a smaller marker set that usually predicts about as well and runs faster.

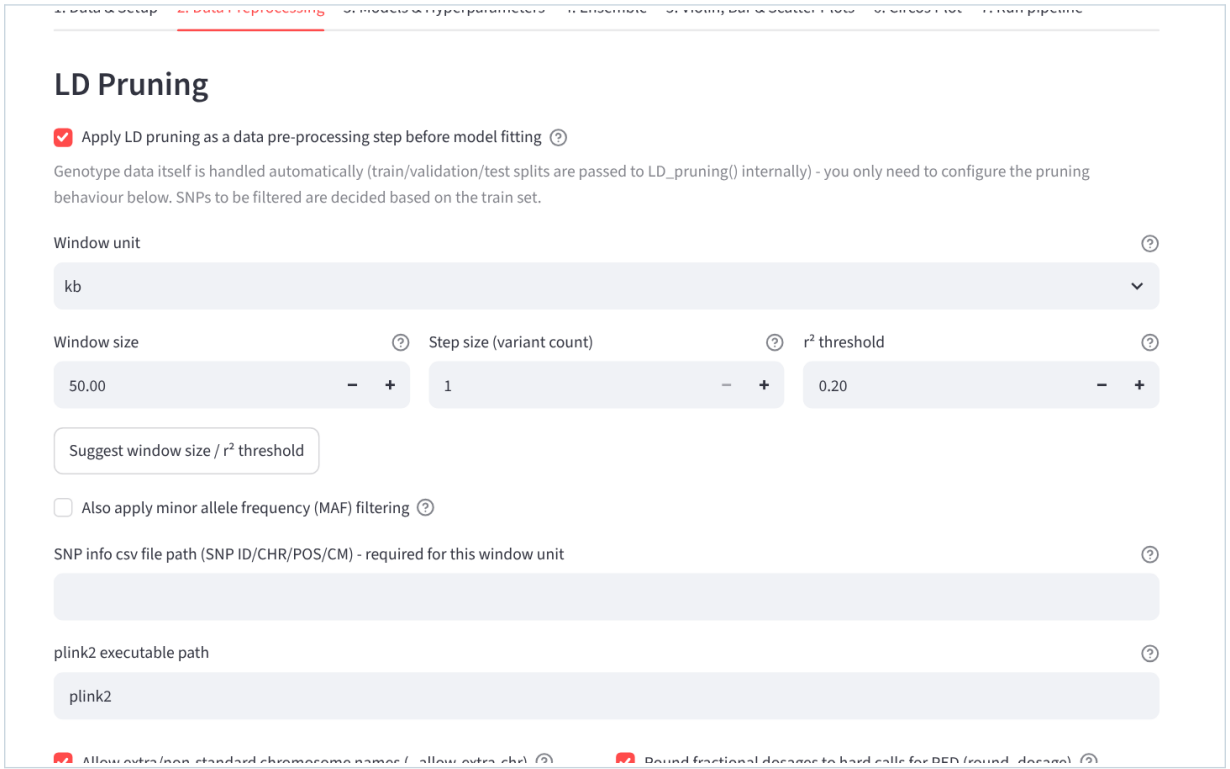


Figure 10. LD pruning settings (shown after ticking the checkbox).

Field	Role
Window unit	*kb*, *variants* (marker count) or *cm*. kb and cM need a SNP info CSV with positions.
Apply LD pruning as a data pre-processing step before model fitting	Switches LD pruning on (off by default). The markers to remove are chosen from each task's training set only.
Window size / Step size (variant count) / r^2 threshold	Sliding window, how far it moves, and the r^2 above which one marker of a pair is removed (default 50, 1, 0.2). A lower threshold removes more markers. **Suggest window size / r^2 threshold** proposes values from your data.
Also apply minor allele frequency (MAF) filtering / MAF threshold	Optionally also removes markers with minor allele frequency below the threshold (default 0.05), i.e. markers where the rarer allele is very rare and carries little information.
SNP info csv file path / plink2 executable path	Marker positions (columns SNP ID, CHR, POS, CM; required for kb and cM windows) and the plink2 program (default `plink2`, i.e. found on the PATH).
Non-human genome (set haploid chromosome count) / Haploid chromosome count (--chr-set)	Ticked by default. **Set the count to your species' number of chromosome pairs** (e.g. maize = 10, wheat = 21); the pre-filled value is **2**, which is almost never right and makes plink2 reject chromosomes 3 and above.
Allow extra chromosome names / round dosages	plink2 compatibility options; keep the defaults unless plink2 reports an error.
Use a custom working directory / Keep intermediate PED/MAP/BED/log files	Where plink2 writes temporary files, and whether to keep them for inspection.
Unmapped-SNP strategy / Unmapped window / Unmapped step	For kb/cM windows: markers without a position are pruned by marker count (*variant_count*, default; window 50, step 5), kept (*skip*) or removed (*drop*).
Generate LD decay plots	Optional diagnostic plots of r^2 against distance, per population and replicate, with advanced plot settings (maximum distance, bin width, marker pairs per chromosome). A plot can be drawn for every Nth combination, and the underlying data kept as CSV. They show how quickly LD fades with distance and help choose the window size.

8.2 RF marker importance filtering

Fits a Random Forest to the training set, ranks markers by how much they help it predict, and keeps only the top markers for all models. This can speed up slow models a lot. Keeping too few markers can lower accuracy for traits controlled by many genes, so compare with an unfiltered run.

Field	Role
Apply RF-based marker importance filtering as a data pre-processing step	Switches the filter on (off by default).
How should the number of markers to keep be specified?	**Percentage of markers** (keep the top 20 % by default) or **Fixed number of markers (M)** (default 100).
Advanced Random Forest settings	Number of trees (200), max features per split, *Limit max tree depth* (off; depth 10 when on), min samples per leaf (1), random seed (0).

Field	Role
Use GPU-accelerated Random Forest for marker selection when available	Experimental. Uses RAPIDS cuML on a GPU; falls back silently to the CPU forest, so it is safe to leave ticked on a CPU-only computer.

8.3 ASReml-R mixed-model adjustment (BLUE/BLUP)

Field trials usually record each line several times (in different plots, blocks, locations or years), and part of the variation comes from the field rather than from genetics. A **linear mixed model** separates these sources and gives one adjusted value per individual: a **BLUE** (genotype treated as a fixed effect) or a **BLUP** (genotype treated as a random effect, shrunk towards the average). EasiGP fits this model to **raw, plot-level** phenotypes and uses the adjusted values for all prediction models. ASReml-R is commercial software with its own paid licence; it is not included in the ready-made images (use installation route B).

Field	Role
Fit an ASReml-R mixed model and substitute converged BLUE/BLUP values before prediction	Switches the stage on. Set **Phenotype file layout = Raw** on Tab 1 first. The section shows a green message once ASReml-R is installed and licensed; otherwise an **Activate ASReml-R licence** box appears.
Genotype column	The column that identifies the individual in the raw file.
Genotype effect	One or more of **BLUEs** (genotype fixed, recommended default), **BLUPs without GRM** (random, independent) and **BLUPs with GRM** (random, with a genomic relationship matrix). Several options are run side by side and compared, each with its own fields below.
Include the entire dataset (every population) when calculating BLUEs/BLUPs	Fits the mixed model on all populations together. Locked on when a spatial/temporal residual structure is used, because ASReml-R then needs the trial's complete layout.
Mask validation/test phenotypes with NA before fitting	Off by default. When ticked, the validation/test records are hidden from the mixed model, so they cannot influence the variance estimates used for their own adjusted values. Tick it for the strictest separation of training and test data.
Two-step refined BLUEs (Holland & Piepho 2024) / Weight second-stage prediction by BLUE reliability	Optional refinements of the BLUE option, with a genomic relationship blend for the first step (default 0.01).
Create a separate genomic relationship matrix for the training, validation and test sets	GRM options only: builds one relationship matrix per set instead of one for everybody.
Marker source for the relationship matrix / Shrinkage towards identity before inversion	Which markers build the relationship matrix and how much it is shrunk towards identity before inversion (default 0.01).
Response unit	*Detect automatically* (recommended), *One value per genotype*, or *One value per genotype x population*: whether a genotype that appears in several populations gets one pooled value or one value per population.
Do the training, validation and test sets share some or all genotypes among them?	between scenario only. Answer *Yes* when the same lines appear in more than one population, so EasiGP can choose a model that handles shared genotypes correctly.

Field	Role
Genotype effect / genomic relationship matrix configuration	The model pattern used for the held-out sets (e.g. BLUP, GBLUP, or BLUEs for training plus a second model for validation/test).
Calculate adjusted prediction accuracy / Use a GRM for the BLUP variance information (BLUEs option)	Adds heritability (classic and Cullis) and adjusted prediction accuracy to `Metric.csv`. For BLUEs, the heritability is borrowed from a companion BLUP fit, without a GRM by default or with one when ticked.
Fixed effects / Random effects	Design columns of the raw file (e.g. Location, Year, Block). Each column may have only one role. As a rule, use fixed effects for a few levels you want to correct for (e.g. Location) and random effects for many levels (e.g. Block).
Variance structures, residual structure, interaction/nesting groups	Optional advanced model terms (e.g. spatial row × column residuals).
Advanced ASReml-R settings	Workspace (4gb) and prediction workspace (4gb), max update rounds (20), *If this task fails to converge* (*Stop the run*, default, or *Skip this task*), strict licence check (on: the whole run stops at the start if ASReml-R is missing or unlicensed), warm start from the previous task, threads (0 = ASReml-R default). Raise the workspace if ASReml-R reports that it ran out of workspace.
ASReml-R syntax (editable)	Preview of the generated model. You may hand-edit it, then click **Commit syntax**.

8.4 Biological prior network

Only used by **GAT_biological_prior_knowledge**; select that model on Tab 3 to unlock this section. The model treats each *gene* as a graph node (built from the markers inside it), connected by a gene-interaction network taken from the literature. In this way existing biological knowledge guides which markers the model lets interact.

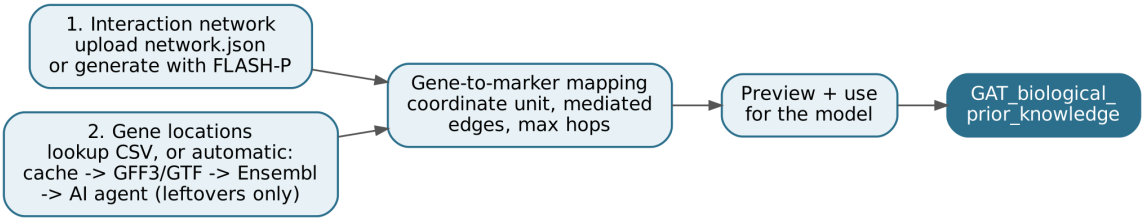


Figure 11. Building the inputs for the biological-prior GAT model.

Field	Role
Species	Passed to FLASH-P and the gene-location lookup (default *Maize*).
Coordinate unit / Include mediated edges / Max hops	bp or cM (must match the gene and marker files); whether genes linked through non-gene nodes (e.g. a hormone) are connected, and through how many.
How many networks do you want to configure?	**One network per phenotype**, or **several independent networks** (default 2; e.g. repeated FLASH-P runs) to see how much the network changes the result. Several networks can also be configured separately per phenotype, and FLASH-P can be run N times in a row from here.
1. Interaction network (JSON) - Where does the network JSON come from?	Give the path of an existing `network.json`, run FLASH-P now (Full version; FLASH-P checkout path, pipeline variant, run mode, claude executable, timeout, permission mode, authentication), or locate the output of an earlier FLASH-P run.

Field	Role
2. Gene locations	Supply a lookup CSV, or build it automatically: cached results, a **species annotation file (GFF3/GTF, strongly recommended)**, Ensembl Plants (free, on by default), then an AI agent for the remaining genes only (agent backend and model, genes per session 25, optional expected genome assembly, WebFetch as a last resort). The table is written to `./Data/_bio_prior_network/gene_location_lookup.csv` by default.
Combine with a data-driven interaction network (RF importance + pairwise Shapley scores)	Off by default. Adds data-driven edges: markers selected by RF filtering that lie outside every gene become their own graph nodes, linked to genes and to each other by their strongest interactions (top 20 % by default). Needs RF marker importance filtering (Section 8.2) to be switched on.
Marker source for other selected models	Which markers the *other* models use: *full_or_filtered* (default; all markers, or the LD-pruned/RF-filtered set), *gene_network* (only the markers inside the network's genes) or *gene_network_plus_rf* (those plus the data-driven markers).
Preview + use for the model	Builds and checks the gene-to-marker mapping and hands it to the model. Unresolved genes are reported and dropped, never guessed. Preview copies can optionally be saved to `./Data/_bio_prior_network`.

9. Tab 3 – Models & Hyperparameters

In short: choose which prediction models to run and, optionally, let EasiGP tune their settings. Tick the models to fit. A settings panel opens for every ticked model. All values have sensible defaults; change them only when you have a reason to. XGBoost and EBM need the optional `xgboost` / `interpret` packages and are greyed out if these are missing.

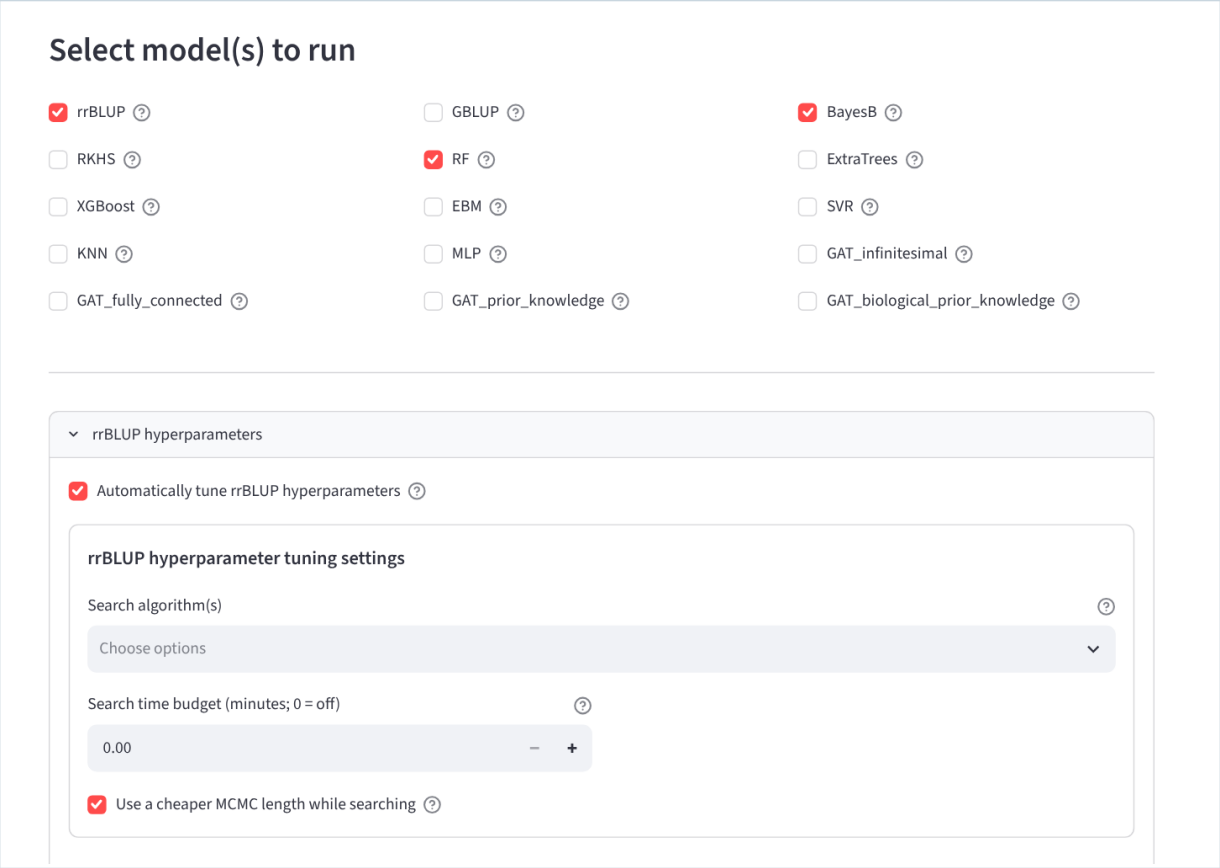


Figure 12. Model selection and the rrBLUP panel with automatic tuning switched on.

9.1 Available models

Model	Family (engine)	Best suited to	Interpretation output
rrBLUP	Linear BLUP (R, BGLR)	Many small additive effects; fast baseline	Marker effects
GBLUP	Relationship-based BLUP (R)	Additive genetics via relatedness	Marker effects
BayesB	Bayesian sparse (R)	A few large-effect markers	Marker effects
RKHS	Kernel (R)	Non-additive, similarity-based	Shapley effects; interactions (estimated)
RF	Random forest (scikit-learn)	Non-linear effects and interactions	SHAP effects; TreeSHAP interactions

Model	Family (engine)	Best suited to	Interpretation output
ExtraTrees	Randomised trees (scikit-learn)	Alternative to RF, less overfitting	SHAP effects; TreeSHAP interactions
XGBoost*	Gradient boosting	Fast, flexible non-linear model	SHAP effects; TreeSHAP interactions
EBM*	Explainable boosting (interpret)	Glass-box model with native pairs	Native pairwise terms
SVR	Support vector regression	Non-linear patterns via kernel	Shapley effects; interactions (estimated)
KNN	Nearest neighbours	Simple similarity-based prediction	Shapley effects; interactions (estimated)
MLP	Neural network (PyTorch)	Complex patterns, larger data	DeepSHAP effects; NID interactions (estimated)
GAT_infinitesimal	Graph attention (self-loops)	Additive-style graph baseline	Marker effects
GAT_fully_connected	Graph attention (all pairs)	Most flexible graph model; slowest	Effects; attention
GAT_prior_knowledge	Graph attention (RF-selected pairs)	Targeted, faster graph model	Effects; interactions; attention
GAT_biological_prior_knowledge	Graph attention over genes	Uses known gene networks (Section 8.4)	Effects; attention

* Optional packages. rrBLUP, GBLUP and BayesB are additive models and do not return marker-pair interactions. “Estimated” interactions (RKHS, SVR, KNN, MLP) are model-agnostic approximations; the tree models, EBM and GAT_prior_knowledge compute their interactions exactly from the fitted model.

Outputs and cost at a glance

Model	Marker effects	Interactions	Attention	Tunable settings	Fitting cost
rrBLUP	always	—	—	4	low
GBLUP	optional (off)	—	—	2	low
BayesB	always	—	—	4	medium
RKHS	optional (off)	optional (off)	—	3	medium
RF	optional (on)	optional (on)	—	5	medium
ExtraTrees	optional (on)	optional (off)	—	5	medium
XGBoost	optional (on)	optional (off)	—	6	medium
EBM	built in	optional (off)	—	4	medium
SVR	optional (off)	optional (off)	—	6	medium
KNN	optional (off)	optional (off)	—	3	low
MLP	always	optional (off)	—	7	high
GAT_infinitesimal	optional (on)	—	—	7	high
GAT_fully_connected	optional (on)	—	yes	7	very high

Model	Marker effects	Interactions	Attention	Tunable settings	Fitting cost
GAT_prior_knowledge	optional (on)	optional (off)	yes	7	high
GAT_biological_prior_knowledge	optional (on)	—	yes	7	high

“optional (on/off)” = a tick box in the model panel and its default. “Tunable settings” = how many of the model’s settings automatic tuning can search (Appendix C lists them). Relative cost refers to fitting only; marker effects and especially interactions can take longer than the fit itself.

9.2 The models in plain words

Figure 3 (Section 2.4) shows the main difference between the families: what they assume about how markers act on the trait.

Model	How it works, simply
rrBLUP	Gives every marker a small effect and adds them up. Robust, fast and a standard reference in plant breeding.
GBLUP	Predicts an individual from how genetically similar it is to the training individuals. Usually gives results very close to rrBLUP.
BayesB	Assumes most markers have no effect and a few have large effects. Good for traits controlled by a few major genes.
RKHS	Like GBLUP, but with a more flexible similarity measure that can pick up gene-by-gene effects.
RF / ExtraTrees	Build hundreds of decision trees (“if marker A = 2 and marker B = 0 then ...”) and average them. Capture interactions naturally.
XGBoost	Builds trees one after another, each correcting the errors of the previous ones.
EBM	Learns one simple curve per marker plus a few marker pairs, so the model itself is easy to read.
SVR	Fits a smooth function that ignores small errors; flexible through a kernel.
KNN	Predicts the average of the k most genetically similar training individuals.
MLP	A neural network: layers of simple units that can learn complex patterns, but need more data and more run time.
GAT models	Neural networks that treat markers (or genes) as points in a network and learn which connections matter (“attention”). The variants differ in which connections are allowed: none (infinitesimal), all (fully connected), those suggested by a Random Forest (prior knowledge), or known gene interactions (biological prior).

Which models should I start with?

A good first set is **rrBLUP + BayesB + RF** (the default): one additive model, one sparse model and one non-linear model. Add **GBLUP** or **RKHS** for a kernel view, and **ExtraTrees** or **XGBoost** for more non-linear diversity. Neural network and GAT models are the slowest; add them once the basic run works, ideally on an HPC. Combining models from different families gives the most useful ensembles (Section 2.5).

9.3 Fields common to most model panels

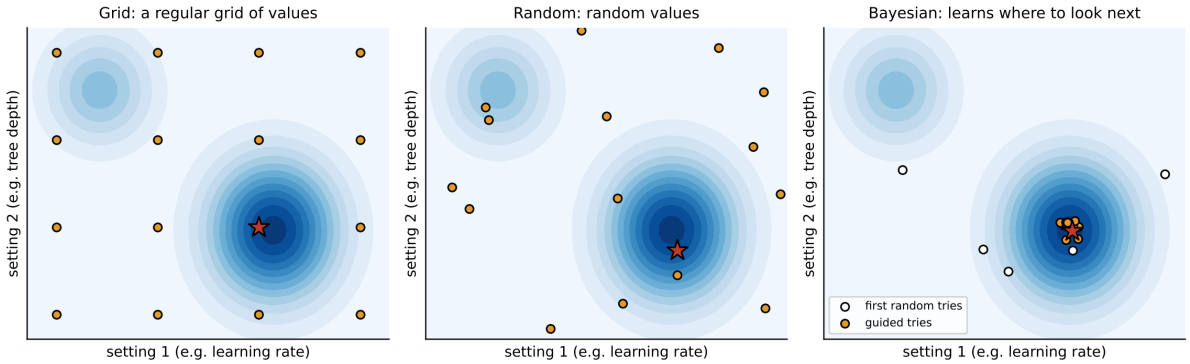
Hyperparameters are settings that control how a model learns (for example the number of trees in a Random Forest or the learning rate of a neural network). The defaults work reasonably well; automatic tuning tries other values and keeps those that predict the validation set best.

Field	Role
Automatically tune <model> hyperparameters	Searches for the best values of the tunable fields on the validation set (score: Pearson r − MSE/phenotype variance), per task. Needs a validation split. Tuning multiplies run time.
Search algorithm(s)	**Grid** (tries a regular grid of values), **Random** (random values), **Bayesian** (learns from earlier tries where to look next; a good default), **Nelder-Mead**, **Powell**. Choosing several gives separately tuned variants (e.g. *RF__Grid*, *RF__Bayesian*).
Grid points per numeric dimension / <algorithm> iterations / Initial random points	How many candidates are tried: Grid uses 5 points per numeric setting (fields with their own step use that step); every other algorithm tries 30 candidates by default; Bayesian starts with 5 random points before it begins to learn.
Search time budget / Time cap per search fit (minutes; 0 = off)	Optional time limits: a total budget for the Bayesian search, and (GAT_biological_prior_knowledge only) a cap per candidate fit.
Use a cheaper MCMC length while searching	rrBLUP, GBLUP, BayesB and RKHS only; on by default. Search candidates run a short MCMC chain; only the final fit uses your full Iteration number and Burn-in. Leave on.
Model-specific fields	e.g. Iteration number and Burn-in (BGLR models, default 12000 / 2000); Tree number (100) for RF; Neuron numbers, Learning rate, Epoch, Batch size (MLP, GAT).
Return marker effect?	Computes marker effects (for the circos rings). Shapley-based effects can be slow; “Number of samples” controls the cost.
Marker effect method	How effects are computed (e.g. closed form for rrBLUP, BayesB and GBLUP; TreeSHAP for tree models; baseline SHAP for RKHS, SVR and KNN). Table C.1 explains each method.
Return marker-pair interactions?	Computes interaction scores for the circos links. Not available for rrBLUP, GBLUP and BayesB.
Max markers considered for interaction search	Restricts the search to the top markers (default 500) to keep it tractable.
Output only the top N% of interactions	**Highlighted in the GUI**: strongly affects both run time and how interpretable the interactions are. *all* keeps everything.
Marker interaction method	e.g. pairwise TreeSHAP (tree models), baseline SHAP (RKHS, SVR, KNN) or NID (MLP). Table C.1 explains each method.

Every setting of every model

Appendix C lists every field of every model panel with its default, the range automatic tuning searches, and a plain-language explanation. Figure 23 there shows what the most common settings do.

How the search algorithms work



Darker blue = better validation score. Each dot = one candidate tried (16 per panel). Red star = best candidate found.

Figure 13. Three ways to search for good settings with the same budget of 16 candidates. Grid and Random spread candidates evenly; Bayesian uses the scores of earlier candidates to concentrate on the promising region.

Nelder-Mead and Powell are classical step-by-step optimisers: they start from the default values and move in the direction that improves the score. They are efficient when there are only a few settings to tune. If you are unsure, choose **Bayesian**.

> **Speed tip**
>
> If you only need accuracies, untick **Return marker effect?** and **Return marker-pair interactions?**: these explanation steps can take longer than fitting the model itself. Keep them on for the models you want to see on the circos plot.

Shared tuning settings (bottom of Tab 3, shown when any model is tuned)

Setting	Default	What it does
Parallelise hyperparameter-search trials	on	Evaluates search candidates on several CPU cores at once. Speed only.
Parallel restarts	on	Runs the independent restarts of Nelder-Mead and Powell in parallel. Speed only.
Batch Bayesian search / Bayesian batch width cap / Batch liar strategy	on / 8 / max	Lets the Bayesian search suggest several candidates per round when several workers are available.
Multi-fidelity screening	off	Screens each batch of candidates cheaply on a subsample before fitting the promising ones fully.
Bayesian domain reduction	auto	Whether the Bayesian search gradually narrows its search box towards promising regions (auto, always, never).
Warm-start tuning across replicates	off	Starts each replicate’s search from the previous replicate’s winner. Faster, but Sequential and Parallel runs may then differ; off keeps them identical.
Tuning selection margin	0.02	The tuned values must beat your own configured values by at least this score, otherwise your values are kept. Protects against “tuned but worse”.
Repeated-resample validation draws	1	Scores each candidate as the average over several inner resamples of the training data. More stable for small data, but slower.

Setting	Default	What it does
Reuse tuned hyperparameters from a pilot run	(blank)	Path to the hyperparameter.csv of an earlier tuned run (e.g. one replicate). Matching tasks skip the search and use the pilot's values.
Fast tuning (and Apply fast tuning preset)	off	Makes every search cheaper without changing the final fits. Its options include freezing run-length fields, **Search space = priority** (search only the ★ settings of Appendix C), an exact closed-form search engine for rrBLUP and RKHS, early stopping after N candidates without improvement, and lean GAT search fits.
Fail fast on missing optional model dependencies	on	Stops a run at the start if a selected model's optional package (xgboost, interpret) is missing.

Acceleration settings (speed only)

This expander appears under the GAT_biological_prior_knowledge panel when that model is selected, but its switches apply to the whole run. Most only change speed: reading PLINK files directly, remembering LD-pruning/RF results across tasks, GPU and batching options for the GAT models, thread limits for worker processes, and fitting a GAT on a separate worker while the CPU models run. The GAT seed settings affect repeatability: **Fix the GAT random seed** (on by default) makes GAT results identical between runs, and **Networks averaged in each FINAL GAT fit** averages several seeded networks for a steadier result. Leave them at their defaults unless you are optimising run time.

10. Tab 4 – Ensemble

In short: choose how the selected models are combined. Selecting several methods lets you compare them in the same run. If you are unsure, keep only the naive ensemble: it needs no validation set and is hard to beat by much.

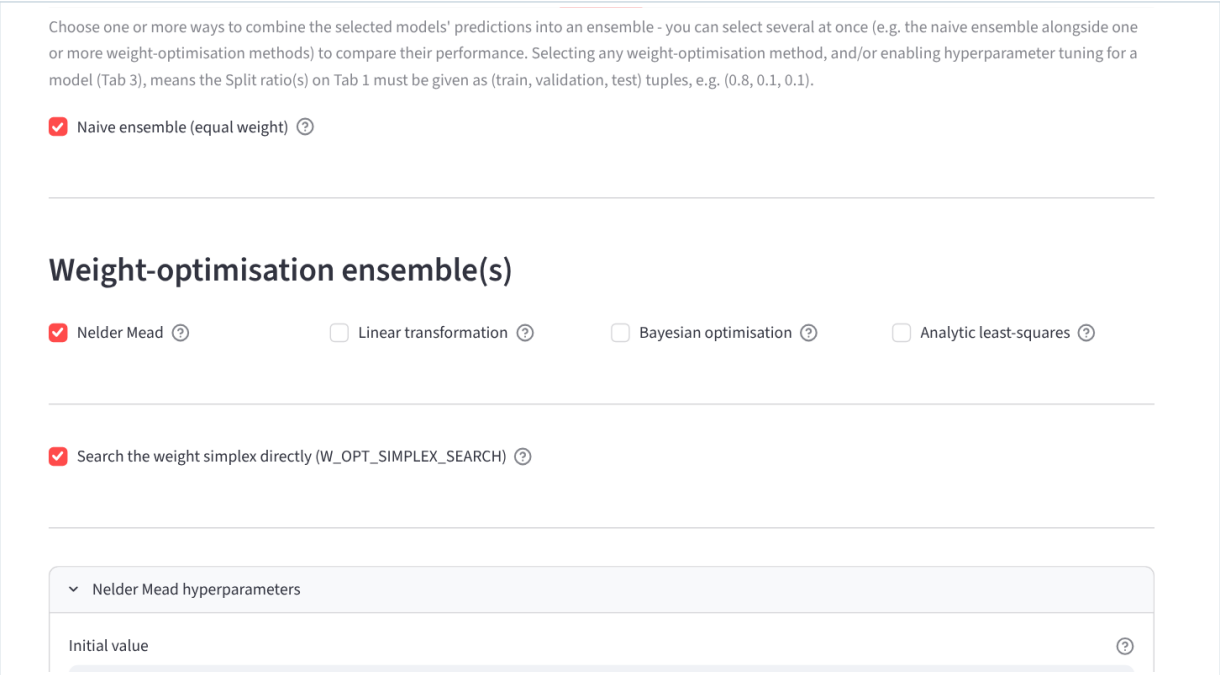


Figure 14. Ensemble choices with Nelder Mead selected.

Option	How weights are found	Notes
Naive ensemble (equal weight)	Simple average of all selected models.	Default; needs no validation set.
Nelder Mead	Gradient-free numerical search.	Simple and generally reliable.
Linear transformation	A small neural network learns the weights (trained 30 times from different starts by default).	Weights are unconstrained (can be negative) unless *Use simplex weights* is ticked.
Bayesian optimisation	Uses past attempts to choose the next weights to try.	Can need fewer attempts; more overhead each.
Analytic least-squares	Exact closed-form least-squares solution.	Milliseconds, nothing to tune, no randomness. A good first weighted method.

How weighted ensembles work. Each model predicts the validation individuals. The method then searches for the weights (e.g. 0.5 × rrBLUP + 0.3 × RF + 0.2 × BayesB) whose combined prediction best matches the measured validation values, and applies those weights to the test set. With small validation sets, learned weights can be noisy, which is why EasiGP offers shrinkage towards equal weights (below).

Field	Role
Search the weight simplex directly (W_OPT_SIMPLEX_SEARCH)	Shown once a weight method is ticked; on by default. Nelder Mead and Bayesian optimisation then search only valid weights (non-negative, summing to 1); the Minimum/Maximum boundary is applied afterwards. Off: the search runs in an unconstrained box, so negative weights are possible when the Minimum boundary is below 0.

Field	Role
<method> hyperparameters	A panel per ticked method. The settings most users meet are: **Minimum / Maximum boundary** (range of each weight; 0 lets a weak model be dropped), **Objective** (*ensemble_mse*, default, or *dpt_ratio*), **Diversity penalty** (optional pull towards equal weights; 0 = off) and **Naive-shrinkage alpha** (1 = weights as found, 0 = the naive ensemble). Every field is listed in Appendix C.11.
More than one hyperparameter-tuning algorithm is in use ... combine:	Appears when a model is tuned with more than one algorithm: *per_method* (one ensemble per algorithm, e.g. ensemble__Grid) or *across_methods* (one combined ensemble).

Remember

Every weight-optimisation method needs a validation split: a (train, validation, test) ratio on Tab 1 (Section 7).

The engine also contains research options (a Super learner, an iterative ensemble and joint hyperparameter/weight optimisation). They are not offered in the GUI and are not covered by this manual.

11. Tab 5 – Violin, Bar & Scatter Plots

In short: chooses which accuracy plots and summary reports are made after fitting. The defaults give the most important ones; Section 16 explains how to read them. Each section has its own **Create ...** checkbox plus Font size, Figure size and Plot quality (DPI; 300 is good for papers). Advanced style expanders are optional.

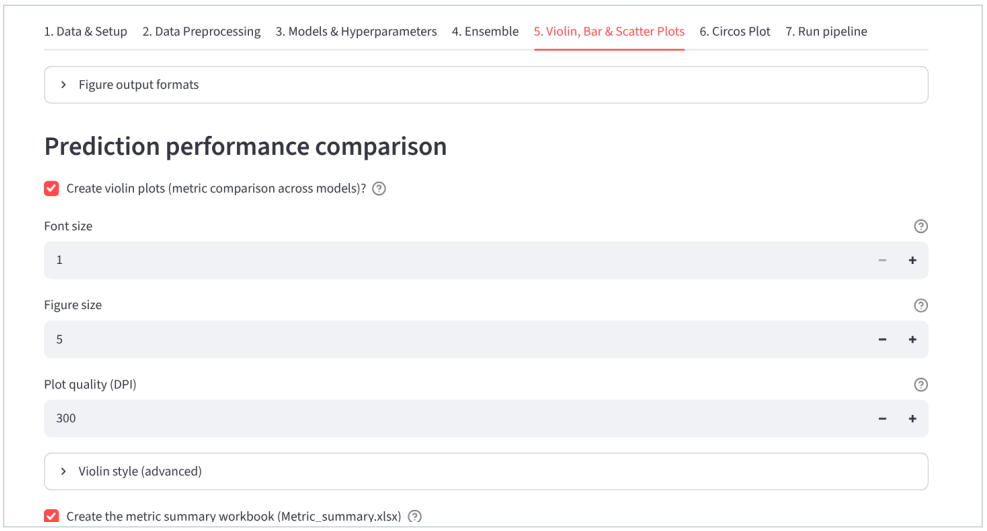


Figure 15. Top of Tab 5.

Section (default)	Output	Notes
Figure output formats	Also save figures as SVG and/or PDF	PNG files are always written. SVG/PDF can be edited and scaled without losing quality.
Prediction performance comparison (on)	Violin plots of Pearson correlation and MSE per model; `Metric_summary.xlsx` (on)	Optional calibration-slope ("dispersion") sheet. *Violin style (advanced)*: style, minimum replicates for a violin (5), colour, reference line, legend, tails, mean ± SE overlay, median labels.
Diversity Prediction Theorem (on)	`Diversity_prediction_theorem.xlsx`; ensemble weight plots `Weight.png` (on)	Explains ensemble error as average model error minus model diversity. Optional `Consensus_stability.csv` (markers and pairs counted as "top": 50 per iteration) and `Interaction_selection_frequency.csv`.
Model pairwise comparison (on)	Scatter-plot matrix comparing models' predictions and marker effects	Optional QTL file (with a QTL window for coordinate-based files); colour dots by population; normalise marker effects (on); readability options (max points per panel 5,000, density display, panel sizing).
Adjusted prediction accuracy (off)	Violin plots	Needs *Calculate adjusted prediction accuracy* on Tab 2 (ASReml-R).

Section (default)	Output	Notes
Spearman correlation (off)	Violin plots	Rank-based accuracy; always computed, plotted only when ticked.
Top-k Coincidence (off)	Curves for k = 5, 10, ..., 30 % of the test set	Share of the true top-k% individuals that a model also ranks in its top-k%. Uses the direction set on Tab 1. Style options: chance baseline, uncertainty bands, Coincidence_curve_summary.csv.
Czekanowski coefficient (off)	Pairwise model scatter-plot matrices	Agreement between two models' top-k% selections; choose the threshold(s).
Pairwise comparison of metrics	Pearson vs Spearman (and adjusted) scatter plots	
Comparison between BLUEs / BLUPs	Each metric compared across ASReml-R options	Needs two or more Genotype-effect options.

12. Tab 6 – Circos Plot

In short: sets up the genome-wide picture of what the models learned. A circos plot shows, for each phenotype and population, the genome as a circle with one ring of marker effects per model, known gene regions, and links between interacting marker pairs. It lets you compare the genetic architecture inferred by different models and spot regions not yet described in the literature. Section 16.7 explains how to read it.

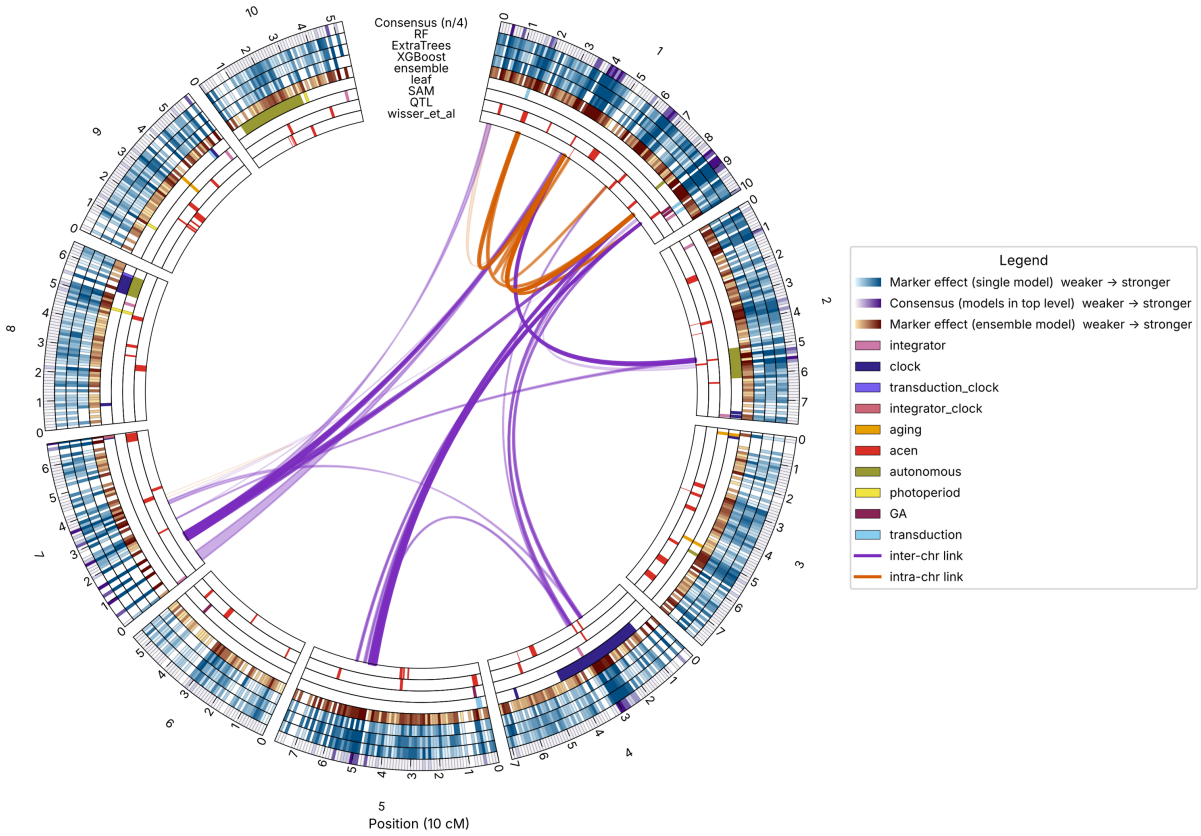


Figure 16. A circos plot produced by EasiGP for the MaizeNAM example (population 1, days2anthesis; RF, ExtraTrees, XGBoost and the naive ensemble, 3 replicates) with the shared legend. Rings from outside in follow the label list at the top: chromosome ticks (10 cM units), the consensus ring (purple), one marker-effect ring per model (blue), the ensemble (brown), then the known gene-region rings from gene_info.csv coloured by category. Links in the centre are the 30 strongest marker-pair interactions of the ensemble (purple between chromosomes, orange within a chromosome).

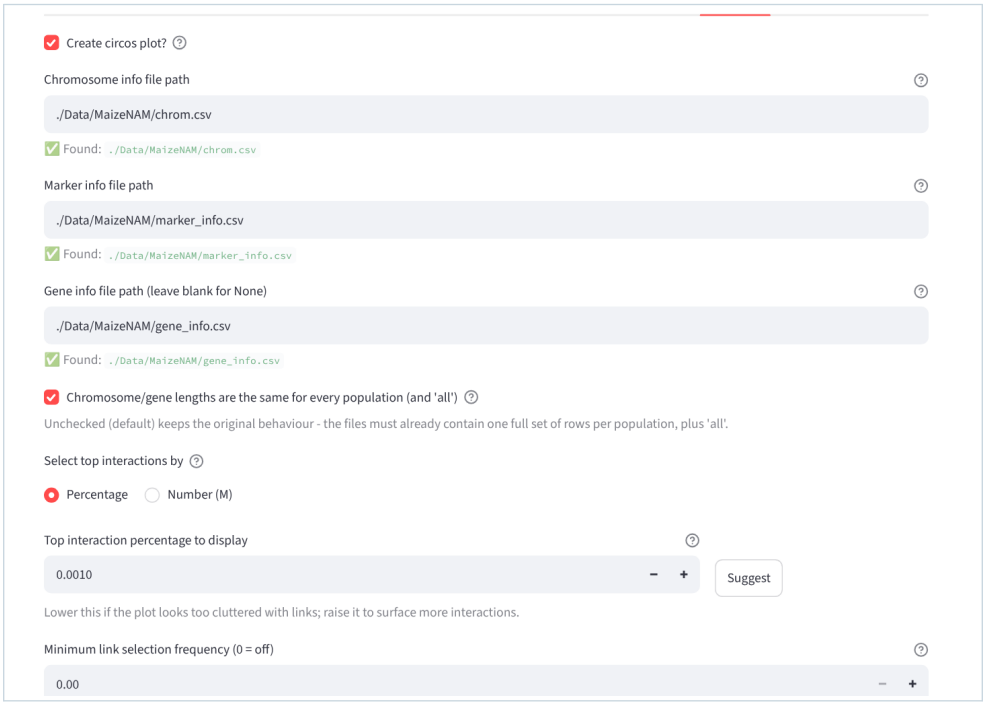


Figure 17. Top of Tab 6; layout values are auto-filled from the MaizeNAM example files.

Field	Role
Create circos plot?	Switches circos plotting on or off (on by default).
Chromosome / Marker / Gene info file path	The `chrom.csv`, `marker_info.csv` and optional `gene_info.csv` (Section 4).
Chromosome/gene lengths are the same for every population (and “all”)	Ticked by default: one set of rows applies to all populations (no population column needed).
Select top interactions by / Top interaction percentage / Top number of interactions (M)	*Percentage* (default 0.001 %) or *Number (M)* (default 75) of the strongest interactions drawn as links. **Suggest** proposes a value from your expected marker count. Lower it if the plot is cluttered.
Minimum link selection frequency	Only draw links selected in at least this share of replicates (0 = off). Raising it keeps only links that appear consistently.
Chromosome coordinate unit	bp or cM.
Space between rings, Start/End angle, Scale, Label sizes	Layout. Auto-filled from your files; type a value to override.
Minimum link opacity	How faint the weakest drawn link may be (the strongest is always opaque).
Averaging window size (WINDOW)	Averages marker effects within windows; 0 plots each marker’s own effect. A window makes dense marker sets easier to read.
Gene location adjustment (GENE_ADJUST)	Widens gene regions so they are visible; does not change their true position.

Field	Role
Links, consensus ring and marker labels (advanced)	Link drawing order, strength scale, colour mode and width (0.6–2.4). **Add an outer consensus ring** (on): counts how many models put each window in their top effect level; *Consensus level* 9 = strongest decile, *Models counted* = all models including ensembles (default) or base models only. **Label the top N markers** (0 = off) writes the strongest markers' (or genes') names and a `marker_effect_top_*.tsv` table.
Highlight interactions from genome regions	Emphasise links touching chosen genes, intervals or chromosomes and fade the rest.
Advanced settings / Edit cytoband colour map	Edge location adjustment, effect ordering, ring-label fit and layout, figure size, maximum seam gap, ring-label length, which interaction orders to draw (k ≥ 3 as hub-and-spoke glyphs); colours for each gene category in `gene_info.csv` (add, change or remove).

13. Tab 7 – Run pipeline

In short: starts the analysis on this computer, or writes the files needed to run it on an HPC cluster. First choose a workflow; only the settings for that workflow are shown.

Option	What happens	Use it for
Option A (recommended for HPC): export config, then submit a job	Writes a configuration JSON and a job script (Slurm, PBS or NCI). Nothing runs in the GUI.	Real analyses on an HPC.
Option B: run now (local)	Runs immediately on the machine running the GUI (**Run pipeline now (local)**). In Parallel Step 1 it becomes **Run single batch now (local test)**, and in Step 2 **Run assemble + plots now (local)**.	Small runs and tests.

What is an HPC job?

A high-performance computing (HPC) cluster is a shared group of powerful computers. You do not run programs on it directly; instead you *submit a job script* that says what to run and how many CPUs, how much memory and how much time it needs. A **scheduler** (Slurm or PBS, depending on your cluster) runs the job when resources are free. An **array job** is many copies of the same job, each working on a different batch of tasks at the same time. Ask your cluster's support team which scheduler, account and partition/queue to use.

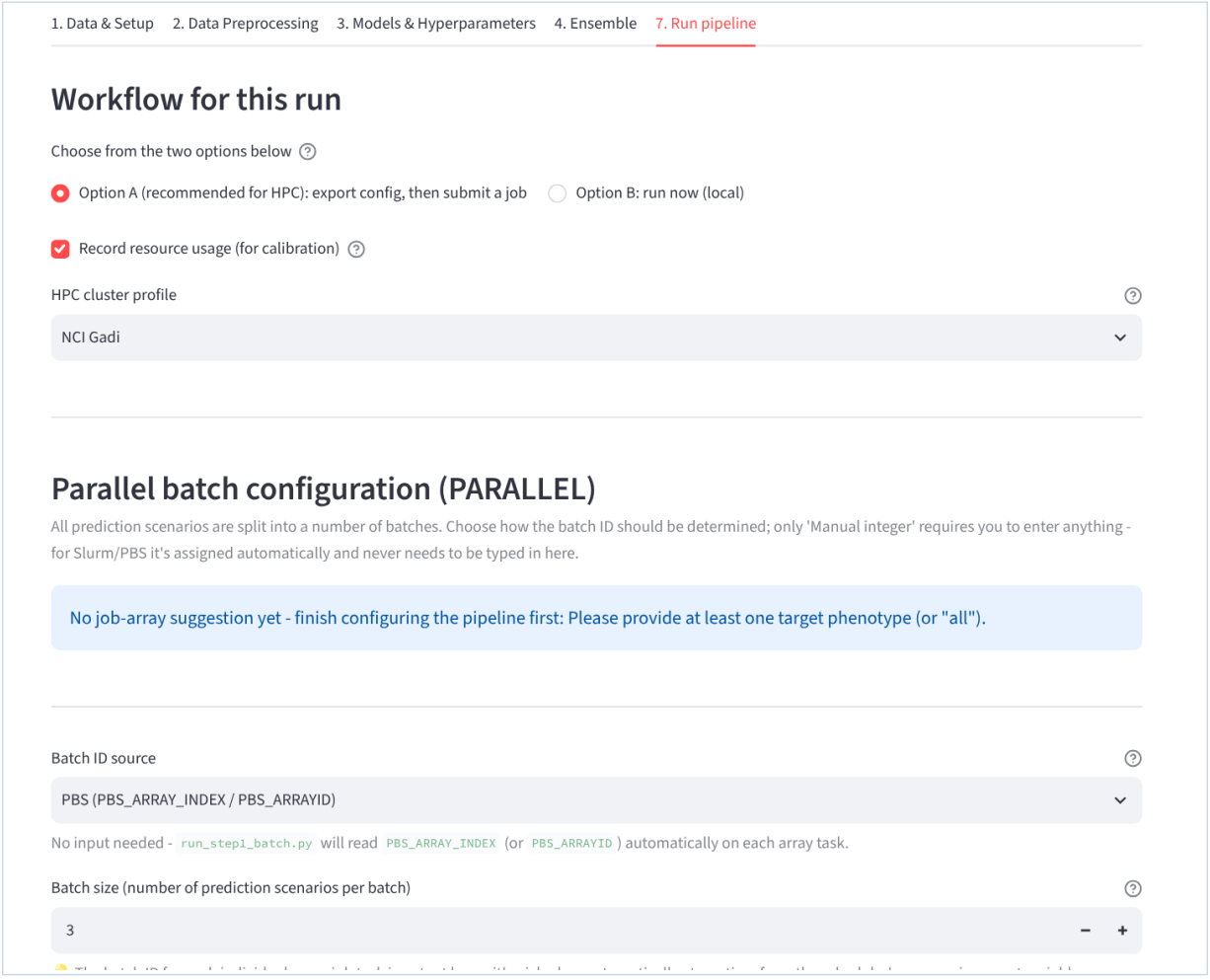


Figure 18. Tab 7 in Parallel / Step 1 with Option A. Suggestion panels fill in once a target phenotype is set.

13.1 Option A fields

Field	Role
Record resource usage (for calibration)	Saves time and memory per task so later resource suggestions are calibrated to your runs. Results are unaffected.
HPC cluster profile	NCI Gadi, UQ Bunya, Generic / other HPC, or Custom (enter your own limits: subjob cap, CPUs per GPU, cores and memory per node, max walltime, GB charged per CPU).
Suggested job-array size / Suggested compute resources	Proposed array layout and CPU/GPU/memory/walltime for your configuration, with an **Advisor engine** choice. Once earlier runs have recorded their resource use, the suggestions are calibrated from them (walltime quantile 0.9, memory quantile 0.95, target walltime per subjob 12 h). **Use these values** copies them into the fields below. Treat them as a starting point.
Batch ID source / Batch ID (Parallel Step 1)	Normally read automatically from the scheduler (SLURM_ARRAY_TASK_ID or PBS_ARRAY_INDEX). A fixed Batch ID is only for testing one batch.
Batch size (number of prediction scenarios per batch)	Number of tasks per array sub-job (default 3). Larger = fewer, longer sub-jobs.
HPC job resource requests	**Job scheduler** (Slurm, PBS, or NCI for Gadi's own array generator), job name, nodes, tasks per node, CPUs per task, memory (e.g. 10G), walltime (HH:MM:SS), partition/queue, account/project code, login shell (on), GPU request (GPUs per task, type, CUDA MPS), QoS (Slurm), storage directive and rerunnable flag (PBS), output/error file names, array start/end index, max concurrently running array tasks, and chunk size for PBS arrays.
Additional setup lines	Lines added to the script before EasiGP runs, e.g. `module load anaconda3` and `conda activate EasiGP`.
Also write a pilot job script / Pilot tasks (K)	Runs K representative tasks first to measure time and memory before a large submission.
Generate and save job files	Writes the files and shows their location and contents. Submit with e.g. `sbatch <job name>.sh` (Slurm) or `qsub <job name>.sh` (PBS).
Convert an existing bash file's line endings (Windows -> Linux)	Fixes scripts saved or edited on Windows, which otherwise fail on the cluster with "/bin/bash^M: bad interpreter" (Section 18.6).

Option A step by step

1. Configure Tabs 1–6, then in Tab 7 choose Option A and your **HPC cluster profile**.
2. Fill in the account and partition/queue your cluster gave you, and add the setup lines that load your EasiGP environment.
3. Optionally write and submit the **pilot job** first, then use its measurements to set walltime and memory.
4. Click **Generate and save job files**; copy the folder to the cluster if the GUI ran elsewhere.
5. Log in to the cluster, go to the EasiGP folder and submit the script (`sbatch` or `qsub`). Check progress with `squeue -u $USER` (Slurm) or `qstat -u $USER` (PBS).

13.2 Parallel mode in practice

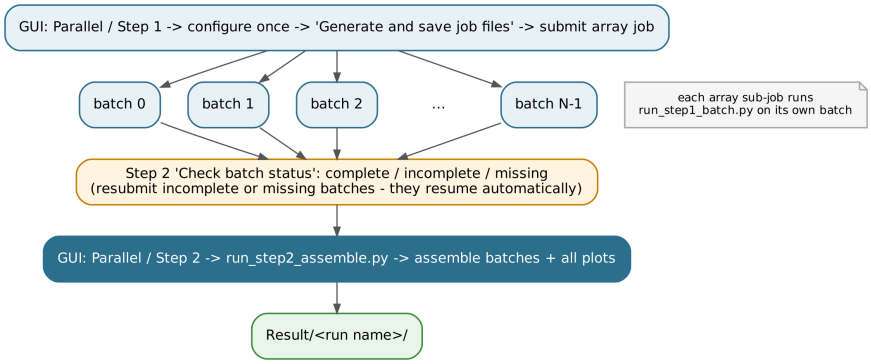


Figure 19. Parallel workflow: Step 1 fits batches as an array job; Step 2 assembles them and plots.

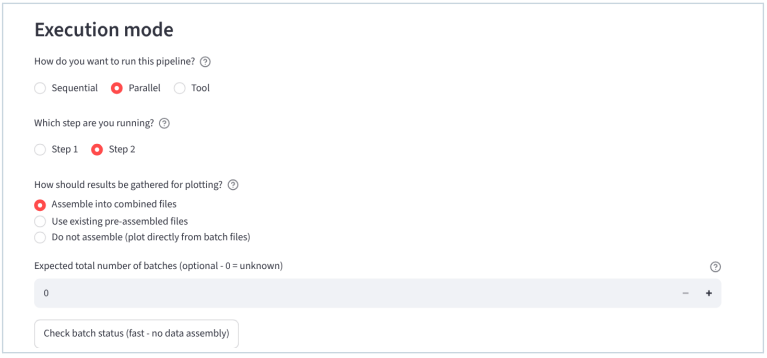


Figure 20. Parallel / Step 2 options at the top of the page.

1. Choose **Parallel → Step 1**, configure Tabs 1–4, then in Tab 7 generate the job files and submit the array job.
2. When the jobs finish, choose **Parallel → Step 2** with the **same Result folder name**. Enter the **Expected total number of batches** (Array end index + 1) and click **Check batch status (fast - no data assembly)** to list complete, incomplete and missing batches.
3. Resubmit incomplete or missing batches with the same configuration: finished tasks are kept and only unfinished ones are re-run.
4. Configure Tabs 5–6 and run Step 2. **How should results be gathered**: *Assemble into combined files* (normal), *Use existing pre-assembled files* (only re-plot), or *Do not assemble* (plot straight from batch files).

> **Tip: changing only the plots**
>
> Plots can be redrawn without refitting: run Step 2 again with *Use existing pre-assembled files* after changing settings on Tabs 5–6 (e.g. font size or circos colours).

14. Tool mode – comparing finished runs

Tool mode reads the saved result files of finished runs and never refits a model. Use it, for example, to compare a run with and without LD pruning, or with different model sets. It has two tabs.

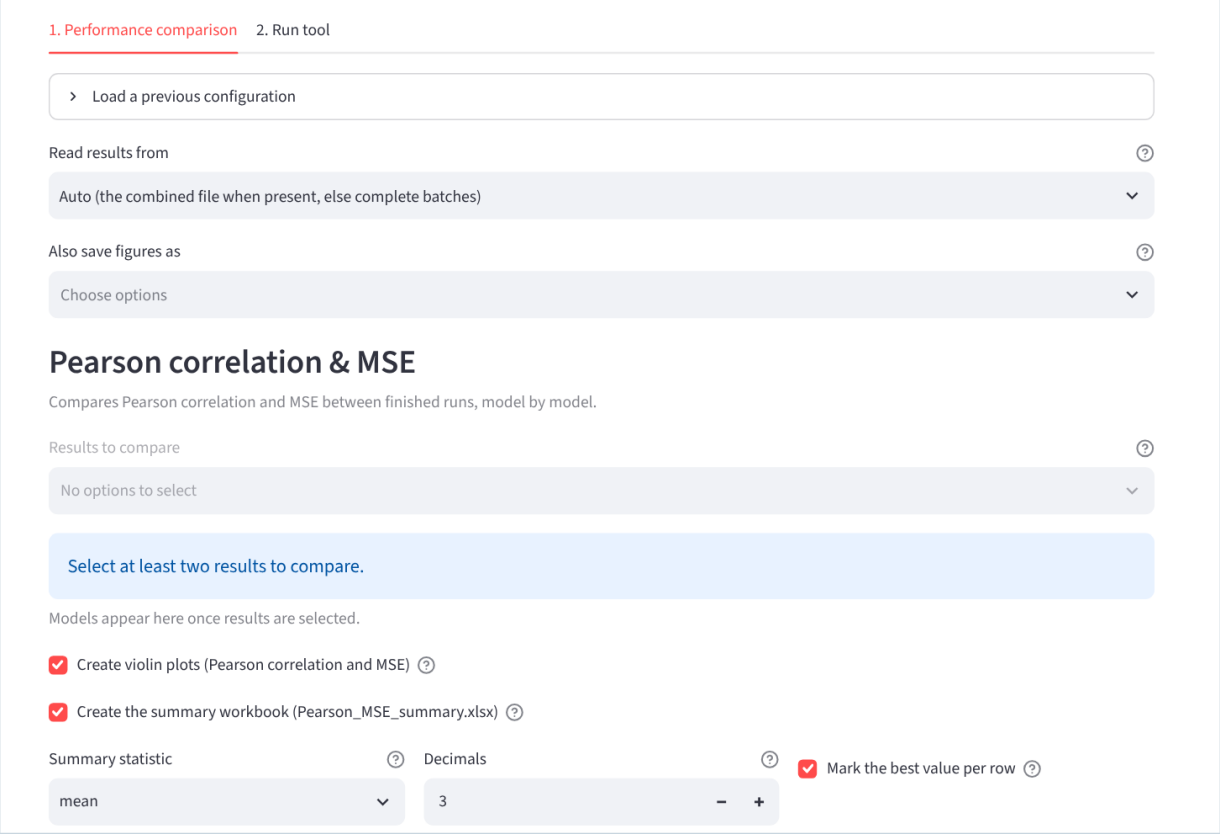


Figure 21. Tool mode, tab 1 (Performance comparison).

Section	Role
Read results from	Auto, combined (assembled) files only, or complete batch files only.
Pearson correlation & MSE / Adjusted accuracy / Spearman	Pick two or more result folders, edit the names shown in figures, choose models; creates violin plots and a summary workbook (statistic, decimals, best value marked).
Comparison of metrics between results	Scatter plots of each metric between pairs of results.
Per-run diagnostics (optional)	Tick **Run per-run diagnostics**, choose the runs and the diagnostics (table below), the prediction table to read (test by default), the number of top markers per iteration (50) and the interval miscoverage. Optional inputs: a genotype CSV (for candidate reliability) and a groups CSV (for within-group accuracy).
2. Run tool	Output folder name inside Result/ (must not be a finished run), then Option A (export job) or Option B (run now).

The per-run diagnostics

Diagnostic (default)	What it tells you
prediction_diagnostics (on)	Calibration slope (are predictions too spread out or too compressed?) and accuracy within groups, e.g. families.
member_importance (on)	How much ensemble weight each model class (additive, sparse, kernel, tree, neural, ...) received.
member_shapley (on)	The Shapley value of each ensemble member: how much it adds to the accuracy of the average, over all combinations of members.
consensus_stability (on)	How stable the top markers (and optionally marker pairs) are across iterations; optionally writes each pair's selection frequency.
candidate_reliability (off)	Flags candidates that are genetically unlike the training set (out of domain) and gives prediction intervals. Needs the genotype CSV; can show what withholding unreliable lines would gain.
class_hierarchy (off)	Compares equal weights per model class with flat equal weights over all models.
pairwise_diversity (off)	Which pairs of models make the same errors (pairs that do add little to an ensemble).

15. Output files

Every run writes into `Result/<run name>/`. Parallel Step 1 writes one file per batch (e.g. `Metric_3.csv`), which Step 2 merges. Large tables may end in `.csv.gz` (compressed; unzip with 7-Zip, or read directly in R or Python). If you only look at three files, look at `Metric_summary.xlsx`, `Pearson correlation.png` and the circos plots.

File	Contents
Metric.csv	One row per model, phenotype, population, ratio and replicate: Pearson correlation, MSE, Spearman, Top-k Coincidence (+ heritability and adjusted accuracy with ASReml-R).
Prediction_result_train / valid / test.csv	Observed and predicted values for each individual. Use the test file to make your own plots or to rank candidates.
Marker_effect.csv	Each model's marker effects (circos effect rings).
Interaction.csv / Attention.csv	Marker-pair interaction scores; GAT attention weights.
Weight.csv	Weight of each model in each weighted ensemble.
hyperparameter.csv	Hyperparameters used or found by tuning.
Basic_stats.csv	Split sizes and number of markers per task. Check it first if results look odd (e.g. very small test sets).
Metric_summary.xlsx / Diversity_prediction_theorem.xlsx	Excel summary reports (plain CSV if openpyxl is missing).
circos_<phenotype>_<population>*.png, circos_legend.png	Circos plots with one shared legend.
Pearson correlation.png, MSE.png, Weight.png, Scatter_plot_*.png, ...	Violin, weight and scatter plots (+ SVG/PDF if requested).
***_config.json**	The configuration used, for repeating the run or reloading it in the GUI. Keep it with your results for reproducibility.

> **Do not delete a run folder between resubmissions**
> Hidden `.checkpoint_*.json` files and partial results let a resubmitted job continue where it stopped.

16. Reading your results

This section explains how to interpret the main outputs, with practical questions to ask of each.

16.1 Accuracy violin plots

`Pearson correlation.png` and `MSE.png` show one "violin" per model. Each violin is the distribution of the metric across all replicates (and populations, if several): its width shows how many replicates gave that value, and the line or dot inside marks the middle. Ask:

- **Which model is highest (Pearson) or lowest (MSE)?** Look at the middle of each violin, and check `Metric_summary.xlsx` for exact means and standard errors.
- **Are the differences real?** If two violins overlap heavily and their means differ by less than about two standard errors, treat the models as equally good. More replicates make this judgement more reliable.
- **Is the ensemble near the top?** An ensemble that matches or beats the best single model, with a narrower violin, is the most dependable choice for selection decisions.
- **Are some models unstable?** A very wide violin, or values near zero in some replicates, suggests the model struggles with this trait or needs tuning.

16.2 Metric_summary.xlsx

The *summary* sheet is a table with traits as rows and models as columns, in blocks of mean and median with their standard errors. The other sheets break the same numbers down by population and split ratio. The standard error tells you how precisely the mean is estimated: smaller is more precise.

16.3 Ensemble weights (Weight.png, Weight.csv)

For each weighted ensemble, the weight of every model (weights add up to 1). A model with a consistently high weight contributed most; a weight near 0 means the ensemble learned to ignore that model. Weights that jump around a lot between replicates usually mean the validation set is too small to tell the models apart; the naive ensemble is then a safer choice.

16.4 Diversity Prediction Theorem

`Diversity_prediction_theorem.xlsx` splits the ensemble's error into two parts: **ensemble error = average error of the models − diversity**, where diversity measures how much the models' predictions differ from each other. An ensemble therefore gains most when its models are individually accurate *and* make different mistakes. If diversity is close to zero, the models are almost interchangeable and combining them adds little; try adding a model from a different family.

16.5 Ranking-based metrics

- **Spearman correlation**: like Pearson, but only the ranking counts. Useful when you will select by rank.
- **Top-k Coincidence**: for k = 5 %, 10 %, ... it shows what share of the truly best individuals the model also placed in its top k %. If you plan to select the best 10 %, read the curve at 10 %.
- **Czekanowski coefficient**: how similar the top-k % selections of two models are (1 = identical choices). Low values mean the choice of model changes which individuals you would select.

16.6 Scatter-plot matrix (model pairwise comparison)

Each small panel compares two models: their predictions for the same individuals, or their marker effects. Points close to the diagonal mean the two models agree. Models that agree closely in predictions add little diversity to an ensemble; models that agree in predictions but not in marker effects reached similar answers by relying on different markers.

16.7 Circos plots

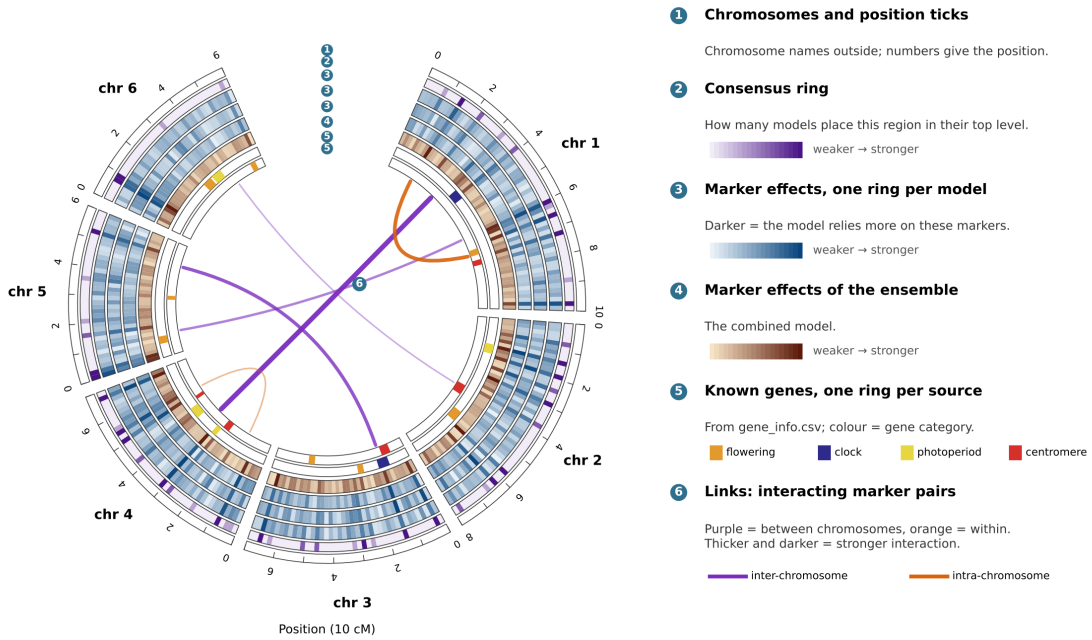


Figure 22. How to read an EasiGP circos plot (schematic with made-up data; compare the real example in Figure 16). The numbered badges in the gap at the top mark each ring. In a real plot that gap holds the ring names, e.g. “Consensus (n/4), RF, ExtraTrees, XGBoost, ensemble, leaf, SAM, QTL”.

Work through a circos plot in this order:

1. **Orient yourself**: chromosomes run clockwise from the top; the outer ring carries position ticks (units shown under the plot, e.g. 10 cM). The list in the gap at the top names every ring from outside in.
2. **Find the strong regions**: marker effects are drawn as colour, not height. In each model ring (blue) and the ensemble ring (brown), darker tiles mark markers the model relied on strongly. In the consensus ring (purple), darker means more models put that region in their top level.

3. **Compare models**: read across the rings at the same angle. A dark region in many rings (and in the consensus ring) is a robust signal; a dark region in only one model may reflect that model's assumptions (e.g. BayesB concentrates effects on few markers, rrBLUP spreads them out).
4. **Check known genes**: the inner white rings show known genes, one ring per source in gene_info.csv, coloured by category. Do the dark regions line up with them? Agreement supports the result; strong regions without a known gene are candidates for follow-up.
5. **Look at the links**: lines in the centre join marker pairs that interact in the model (purple between chromosomes, orange within one); thicker, more opaque links are stronger. Many links meeting at one region suggest a "hub" that interacts with several others.

> **Making circos plots clearer**
>
> Too many links: lower **Select top interactions by** or raise **Minimum link selection frequency**. Noisy rings: set an **Averaging window size**. Genes too thin to see: increase **Gene location adjustment**. Redraw without refitting with Parallel Step 2 (*Use existing pre-assembled files*).

17. Recommended settings by goal

These are suggested starting points, not rules. Adjust them to your data size and computing resources.

Goal	Tab 1	Tab 3 models	Tab 4 ensembles	Other
Quick test of a new dataset	One trait, split 0.8, 2–3 replicates	rrBLUP, RF	Naive	Sequential, Option B
Standard accuracy comparison	Split 0.8, ~10 replicates	rrBLUP, GBLUP, BayesB, RKHS, RF, XGBoost	Naive	Interactions off to save time
Best possible accuracy	Split (0.8, 0.1, 0.1), 10–20 replicates	Diverse set incl. a neural or GAT model; tuning on (Bayesian)	Naive + Analytic least-squares	HPC, Parallel mode
Genetic interpretation	Split 0.8, ≥10 replicates	rrBLUP, BayesB, RF, ExtraTrees, GAT_prior_knowledge	Naive	Effects and interactions on; gene_info.csv supplied; circos link frequency filter
Across-populati on prediction	Scenario *between*	rrBLUP, GBLUP, RF	Naive	Expect lower accuracy for distant populations
Raw field-trial data	Raw layout	Any	Any	ASReml-R (Tab 2)

17.1 Managing run time

Run time grows with the number of tasks (traits × populations × split ratios × replicates), the number of individuals and markers, the models chosen and the explanation settings. From fastest to slowest, roughly: rrBLUP, GBLUP, KNN, RF/ExtraTrees, XGBoost, BayesB/RKHS, SVR, EBM, MLP, GAT models. To speed things up:

- Untick marker effects and interactions for models you will not show on the circos plot.
- Lower **Output only the top N% of interactions**, **Max markers considered for interaction search** and Shapley sample numbers.
- Reduce markers with LD pruning or RF filtering (Tab 2).
- Use the **Fast tuning** preset, or tune only the models that benefit most.
- Use Parallel mode on an HPC, with a pilot job to size the requests.

18. Troubleshooting and FAQ

Most problems fall into a few groups: the computer cannot run Docker, the R part of EasiGP cannot be reached through **rpy2** (the bridge between Python and R), the Python environment is incomplete, or an HPC job is set up incorrectly. Search this chapter for a few words of the message you see.

18.1 Finding the real error message

An error shown in the browser is often only the last line. The full message is in one of these places:

- **The terminal** that runs EasiGP (or the Docker/Apptainer terminal). Scroll up to the first line that starts with `Traceback`, `Error` or `[GP] ERROR`. The **last** line of a Python traceback names the error; the lines just above it say where it happened.
- The **Pipeline log** expander at the bottom of Tab 7 (Option B runs).
- On an HPC, the job's output and error files (e.g. `<job name>_<id>.output` / `.error` or `slurm-<id>.out`). Batch jobs also print a Python stack trace when R or a library crashes without a normal error message.

When a task fails, EasiGP prints `[GP] ERROR: task 3/40 (population=..., phenotype=..., ratio=..., replicate=...) failed: ...` followed by the traceback, saves every task finished so far, and prints `[GP] Partial results saved under './Result/<name>/'`. After fixing the cause, start the same configuration again with the same result folder name: it resumes from the failed task.

Lines starting with `R[write to console]`, BGLR progress lines and most `UserWarning` / `FutureWarning` lines are normal messages, not errors.

18.2 General problems

What you see	Cause and what to do
A file path shows no green "Found" message	The path is wrong or relative to a different folder. Relative paths start from the EasiGP folder; use the full path if unsure. With Docker, use the path *inside* the container, e.g. `/workspace/data/geno.csv`, not `C:\Users\...` (Section 18.3).
Run blocked: a validation split is needed	Weight optimisation or tuning is on. Use a tuple split such as (0.8, 0.1, 0.1), or tick **Use a validation set** (between).
Model checkbox replaced by "unavailable - install the optional ... package"	Install `xgboost` or `interpret` in the EasiGP environment and restart the GUI.
Fewer individuals than expected / tasks skipped	IDs differ between the genotype and phenotype files, genotype rows contain missing values (CSV input), or fewer individuals than **Minimum data points required per iteration**. Check `Basic_stats.csv`.
Target phenotype list is empty	The phenotype file was not found, or its trait columns are not after the first two columns.
`could not convert string to float` **or** `non-numeric argument`	A trait or genotype column contains text such as "NA ", "-", "." or a unit. Leave missing values as empty cells and remove text from numeric columns.

What you see	Cause and what to do
A gene does not appear on the circos plot	Chromosome names differ between files, or the gene's phenotype column does not match the trait name.
Circos plot is empty or markers are missing	Marker names in `marker_info.csv` do not match the genotype column names exactly (case and spelling), or units (bp/cM) differ between files.
Circos plot is cluttered	Lower **Top interaction percentage** or use **Number (M)**; raise **Minimum link opacity**.
LD pruning fails, or plink2 drops chromosomes	Check the plink2 path, and set **Haploid chromosome count** to your species' number (the pre-filled value is 2).
`EasiGP code changed since this window started`	The code was updated while the GUI was open; stop it (Ctrl+C) and start it again.
`PermissionError: [WinError 32] The process cannot access the file because it is being used by another process`	A result file (often an .xlsx or .csv) is open in Excel, or the folder is being synchronised by OneDrive/Dropbox. Close the file, or keep EasiGP and its results outside synchronised folders.
Run is very slow	See Section 17.1.

18.3 Docker and Apptainer

Most first-time Docker problems are about Docker Desktop itself, the shape of the `docker run` command, or where files are saved. A quick self-check: `docker version` should list both a Client and a Server, and `docker run --rm hello-world` should print "Hello from Docker!".

Installing and starting Docker

What you see	Cause and what to do
`docker: command not found` / `'docker' is not recognized as an internal or external command`	Docker is not installed, or the terminal was opened before installation. Install Docker Desktop, then open a *new* terminal.
`Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?` / `error during connect: ... dockerDesktopLinuxEngine: The system cannot find the file specified`	Docker Desktop is not running yet. Start it and wait until it shows "Engine running" (this can take a minute after a reboot). On Linux without Docker Desktop: `sudo systemctl start docker`.
`permission denied while trying to connect to the Docker daemon socket`	Linux only: your user is not in the docker group. Run `sudo usermod -aG docker $USER`, then log out and in again (or prefix commands with `sudo`).
`WSL 2 installation is incomplete` / `WSL needs updating` **(Windows)**	Open PowerShell **as administrator**, run `wsl --install` (first time) or `wsl --update`, restart Windows, then start Docker Desktop.
`Virtualization support not detected` / `Hardware assisted virtualization and data execution protection must be enabled in the BIOS`	Enable virtualisation (Intel VT-x or AMD-V/SVM) in the computer's BIOS/UEFI settings. On a managed work laptop, ask your IT department; installing Docker Desktop also needs administrator rights.
Docker Desktop keeps restarting or is very slow on Windows	Give it more resources (Settings → Resources), update Windows and WSL (`wsl --update`), and restart. Corporate antivirus or VPN software can also interfere.

Downloading the image (docker pull)

What you see	Cause and what to do
`manifest for shunichirot/easigp:latest not found` / `manifest unknown`	There is no “latest” tag. Always give the exact tag from the Docker Hub tags page, e.g. `shunichirot/easigp:<version>_light`.
`no matching manifest for linux/arm64/v8 in the manifest list entries`	Apple Silicon Mac (M1–M4): the image is built for Intel/AMD (linux/amd64). Add `--platform linux/amd64` to both `docker pull` and `docker run`, and in Docker Desktop tick “Use Rosetta for x86_64/amd64 emulation on Apple Silicon”. It runs, but more slowly, and without GPU support.
`The requested image's platform (linux/amd64) does not match the detected host platform`	An expected warning when running with emulation on an Apple Silicon Mac. It can be ignored.
`no space left on device` **during the download**	The image is several gigabytes, and larger when unpacked. Free space with `docker system prune` (removes stopped containers and unused images) and raise *Disk image size* in Docker Desktop (Settings → Resources).
`toomanyrequests: You have reached your pull rate limit`	Docker Hub limits anonymous downloads. Create a free Docker Hub account, run `docker login`, and try again.
`TLS handshake timeout` / `net/http: request canceled while waiting for connection` / `i/o timeout`	Network or proxy problem. Retry; on a university or company network set the proxy in Docker Desktop (Settings → Resources → Proxies), or try another network.

Running the container (docker run)

What you see	Cause and what to do
`Bind for 0.0.0.0:8501 failed: port is already allocated`	Another container or program already uses port 8501, often an EasiGP container you started earlier. List containers with `docker ps` and stop it with `docker stop <container id>`, or use another port: `-p 8502:8501` and open http://localhost:8502.
`docker: invalid reference format` / `repository name must be lowercase`	Docker could not read the image name. Common causes: a typo or capital letters in the image name or tag; a missing tag; a folder path with spaces that is not in quotes; or the README placeholder `YOUR TOKEN` (with a space) left in the command. Put every `-v` path in double quotes and replace all placeholders.
In PowerShell or Command Prompt, the second line of the command is run on its own or gives `Missing expression after unary operator '--'`	The trailing \ line continuation only works on macOS/Linux. In PowerShell put the command on one line (or end lines with a backtick `` ` ``); in Command Prompt end lines with ^ and use `%cd%` instead of `${PWD}`.
`the input device is not a TTY`	You are using Git Bash or MinTTY on Windows. Use PowerShell instead, or prefix the command with `winpty`.
`invalid mode` / `invalid volume specification` / `includes invalid characters for a local volume name`	The `-v` option is malformed. Its form is `"<absolute folder on your computer>:/workspace/data"`. On Windows use `"${PWD}\data:/workspace/data"` in PowerShell or `"%cd%\data:/workspace/data"` in Command Prompt.
The container stops immediately, or `Error: Invalid value: File does not exist: main_app.py`	The command after the image name is wrong, or a folder was mounted over EasiGP’s own folder (`/workspace/EasiGP`), hiding the code. Mount only `/workspace/data` and `/workspace/EasiGP/Result`.

What you see	Cause and what to do
`Killed`**, exit code 137, or the GUI shows "Connection error" in the middle of a run**	The container ran out of memory. Raise Docker's memory limit: Docker Desktop → Settings → Resources (on Windows with WSL 2, add `[wsl2]` and `memory=16GB` to `%UserProfile%\.wslconfig`, then run `wsl --shutdown`). Otherwise reduce the markers (LD pruning, RF filtering), the number of models run together or the worker settings.
`OSError: [Errno 28] No space left on device` **mentioning** `/dev/shm`**, or** `Bus error` **/** `DataLoader worker ... killed by signal`	Docker's shared memory is only 64 MB by default, too small for EasiGP's parallel workers and PyTorch. Start the container with `--shm-size=8g` (Section 3.1).
Runs are much slower than expected	Docker may be limited to a few CPU cores (Docker Desktop → Settings → Resources). On an Apple Silicon Mac the amd64 emulation is slower; for large analyses use a Linux computer or an HPC.

The browser cannot reach the GUI

What you see	Cause and what to do
`This site can't be reached` **/** `ERR_CONNECTION_REFUSED` **at http://localhost:8501**	Wait until the terminal shows "You can now view your Streamlit app in your browser". Then check that the command contains `-p 8501:8501` and `--server.address=0.0.0.0`, and that the container is still running (`docker ps`).
The terminal shows a "Network URL" or "External URL" (e.g. http://172.17.0.2:8501 or http://0.0.0.0:8501) that does not open	Those addresses are inside the container. On your own computer always open `http://localhost:8501` (or the port you chose).
The page stays on "Please wait..." or keeps reconnecting	A VPN, proxy or browser extension is blocking the connection Streamlit uses. Try another browser, `http://127.0.0.1:8501`, or disconnect the VPN.

Files, folders and results

What you see	Cause and what to do
Results or settings are gone after closing Docker	Every `docker run` starts a fresh container; only mounted folders are kept. Always mount the Result folder (`-v "...Result":/workspace/EasiGP/Result`). If the old container still exists, copy the results out: find it with `docker ps -a`, then `docker cp <container id>:/workspace/EasiGP/Result ./Result_recovered`.
The mounted folder looks empty inside the container	The folder path on your computer is misspelt: Docker then silently creates a new, empty folder. Check the path. On macOS, the folder must be allowed under Docker Desktop → Settings → Resources → File sharing (/Users is shared by default).
`Permission denied` **when EasiGP writes into the mounted Result folder (Linux)**	The container user cannot write to your folder. Run `chmod -R a+rwX Result` on your computer, or start the container with `--user $(id -u):$(id -g)`.
Result files are owned by root and cannot be deleted (Linux)	Files written by the container belong to its user. Run `sudo chown -R $USER Result`.
A Windows path such as C:\Users\me\data\geno.csv is "not found" in the GUI	The GUI runs inside the container, which only sees mounted folders. Use `/workspace/data/geno.csv` (and keep the file in the mounted data folder).

What you see	Cause and what to do
Docker uses a lot of disk space	Old containers and images accumulate. `docker container prune` removes stopped containers; `docker image prune -a` removes images not in use (you will have to download EasiGP again).

GPU in Docker

What you see	Cause and what to do
`could not select device driver "" with capabilities: [[gpu]]`	Docker cannot see the GPU. Linux: install the NVIDIA Container Toolkit and restart Docker. Windows: update the NVIDIA driver and use Docker Desktop with WSL 2. macOS has no CUDA GPU support; leave out `--gpus all`.
The GPU is not used although `--gpus all` **is given**	Check with `docker run --rm --gpus all shunichirot/easigp:<tag> nvidia-smi`. If that fails or shows an old driver, update the NVIDIA driver on the host. EasiGP falls back to the CPU, so results are still produced.

Full version: Claude Code and FLASH-P

What you see	Cause and what to do
`Could not start 'claude' - [Errno 2] No such file or directory`	The claude program is not on the PATH of the user running EasiGP. Enter its full path in the **claude executable** field (find it with `which claude`).
Authentication fails, or FLASH-P stops immediately	The token was not passed into the container (`-e CLAUDE_CODE_OAUTH_TOKEN=...` or, on Apptainer, `APPTAINERENV_CLAUDE_CODE_OAUTH_TOKEN`), it has expired, or an old `ANTHROPIC_API_KEY` takes precedence. Use **Test Claude Code connection** on Tab 2 and choose the matching **Authentication** option.
Claude Code refuses to start with permission mode *bypassPermissions*	It refuses this mode when running as root, as inside many containers. Use *auto* or *dontAsk*.

Apptainer on an HPC

What you see	Cause and what to do
`apptainer: command not found`	Load it first: `module load apptainer` (some clusters call it `singularity`). Check your cluster's documentation.
`disk quota exceeded` **or** `no space left on device` **while building**	Apptainer caches the download in your home folder, which is usually small. Before building, run `export APPTAINER_CACHEDIR=/scratch/user/$USER/.apptainer_cache` and `export APPTAINER_TMPDIR=/scratch/user/$USER/tmp` (create the folders first).
`dial tcp: lookup registry-1.docker.io: no such host` **or other network errors while building**	The node has no internet access. Build on a login node, or use the .tar route of Section 3.2.
`Read-only file system` **when EasiGP writes results**	Start Streamlit from the sandbox folder on your scratch space, as in Section 3.2 (`cd /scratch/user/$USER/EasiGP/workspace/EasiGP/`), so Result/ is written to writable storage.
Your data folder is not visible inside the container	Home and scratch are usually visible automatically; other file systems must be bound: `apptainer run --bind /path/to/data /scratch/user/$USER/EasiGP`.

What you see	Cause and what to do
`channel 2: open failed: connect failed: Connection refused` **in the SSH tunnel**	Streamlit is not (yet) running on that compute node, the node name is wrong, or the port differs. Check the node name with `squeue --me`.
`bind [127.0.0.1]:8501: Address already in use` **on your own computer**	Another tunnel or program uses port 8501 locally. Use `ssh -N -L 8502:<node>:8501 ...` and open http://localhost:8502.
`setlocale: LC_ALL: cannot change locale` / `Setting LC_CTYPE failed`	Harmless. To silence it: `export LANG=C.UTF-8 LC_ALL=C.UTF-8` before starting.
EasiGP is killed after a few minutes on the login node	Login nodes limit memory and time. Always start an interactive job (or a virtual desktop) first.

18.4 R and rpy2

rrBLUP, GBLUP, BayesB and RKHS (and ASReml-R) run in R. Python talks to R through the **rpy2** package, which loads R into the same program and converts tables between pandas and R. For this to work, three things must match: rpy2 must find **one specific R**, that R must be the version rpy2 was installed for, and that R must contain the packages EasiGP uses (**BGLR, stringr, data.table, dplyr** and **iml**). The conda environments and Docker images provide exactly such a matching set; almost every rpy2 error comes from a *different* R being picked up, or from packages upgraded by hand.

Check your R set-up in one minute

With the EasiGP environment activated (or inside the container), run:

```
R --version
Rscript -e "R.home(); .libPaths(); library(BGLR); library(iml)"
python -c "import rpy2.robjects as ro; print(ro.r('R.home()'))"
python -m rpy2.situation
```

The R home printed by Python must be the same as the one printed by Rscript, and both library() calls must succeed. The last command is rpy2's own diagnostic report; include its output when asking for help.

rpy2 cannot find or start R

What you see	Cause and what to do
`ModuleNotFoundError: No module named 'rpy2'`	The EasiGP environment is not activated (`conda activate EasiGP`), or the GUI was started from another Python.
Messages mentioning `R_HOME`**, e.g.** `R_HOME not defined`**,** `r_home is None` **or** `R_HOME must be set in the environment or Registry`	rpy2 could not find R. Activate the environment and keep **Auto-detect R installation** ticked. If that still fails, untick it and enter the R home of the environment: `$CONDA_PREFIX/lib/R` (Linux/macOS) or `%CONDA_PREFIX%\Lib\R` (Windows). `Rscript -e "R.home()"` prints the right folder.
`OSError: cannot load library '...\R.dll': error 0x7e` **(or 0xc1) on Windows**	R's DLLs cannot be loaded: the path points to a wrong or 32-bit R, or R's `bin\x64` folder is not found. Use a 64-bit R that matches rpy2 (the environment's own R), enter its folder in **R installation path** (EasiGP adds `bin\x64` to the PATH itself) and restart the GUI.
`libR.so: cannot open shared object file: No such file or directory` **(Linux)**	The R found is not built as a shared library (some system or module R builds). Use the environment's R (conda r-base is built correctly) by keeping auto-detect ticked in an activated environment.

What you see	Cause and what to do
`UserWarning: Error importing in API mode ... Trying to import in ABI mode`	A normal rpy2 message on Windows. It can be ignored.
`R is not initialized by the main thread`	Harmless: Streamlit runs EasiGP in a separate thread. The only effect is that Ctrl+C may not stop a running R fit at once; wait, or close the terminal.

R packages and versions

What you see	Cause and what to do
`Error in library(BGLR) : there is no package called 'BGLR'` **(or** `'iml'`**,** `'dplyr'`**,** `'data.table'`**,** `'stringr'`**)**	rpy2 is using an R that does not have EasiGP's packages, usually a separately installed system R. Tick **Auto-detect R installation** and start EasiGP from the activated environment. If you deliberately use your own R, install the packages there: `install.packages(c("BGLR","stringr","data.table","dplyr","iml"))`.
`package 'X' was built under R version 4.y.z`	A warning, usually harmless. If it is followed by a load error, packages from another R version are being mixed in (next row).
`namespace 'X' 0.1 is being loaded, but >= 0.2 is required` / `package or namespace load failed` / `undefined symbol`	R is loading packages from a personal library made for another R version. `Rscript -e ".libPaths()"` should list only the environment's library. Remove `R_LIBS` / `R_LIBS_USER` settings (e.g. in ~/.Renviron or ~/.bashrc), or recreate the environment.
R model results look wrong, or `AttributeError` / `TypeError: string indices must be integers` / `KeyError: 'r_effect'` **after an R model finished**	rpy2, pandas or numpy were upgraded by hand (e.g. `pip install -U`). EasiGP is tested with the versions pinned in `Environments/*.yml` (rpy2 3.5.11, pandas 2.2.3, numpy 2.1.3 on Linux). Restore them with `conda env update -f Environments/environment_<os>.yml --prune`, or recreate the environment.
`NotImplementedError: Conversion rules for `rpy2.robjects` appear to be missing`	rpy2's pandas conversion was not set up in the thread that called R. EasiGP sets it up at every entry point, so this points to an unsupported rpy2 version or a stale GUI session. Restart the GUI; if it persists, restore the pinned rpy2 version (row above).

Large data, crashes and memory

What you see	Cause and what to do
`Error: protect(): protection stack overflow`	R ran out of its internal "protect" stack while receiving a very wide marker table. EasiGP raises R's limit to the maximum (--max-ppsize=500000) when it starts, and sends tables with more than 100,000 markers to R in blocks. If you also see `NOTE: R is already initialised - can no longer raise its --max-ppsize`, restart the GUI or job so the limit is applied before R starts. You can make block transfer start earlier by setting the environment variable `EASIGP_R_DF_CHUNK_ABOVE=20000` before starting EasiGP, and reduce markers with LD pruning or RF filtering.
`Error: cannot allocate vector of size ... Gb`	R ran out of memory. GBLUP and RKHS build an individuals × individuals matrix; rrBLUP and BayesB hold the full marker table. Use a machine (or job) with more memory, fewer markers, or fewer models running at the same time (fewer workers).

What you see	Cause and what to do
`Segmentation fault`, `*** caught segfault ***`, `corrupted double-linked list`, `free(): invalid pointer`, **or the program ends with no Python traceback**	A crash inside R or another compiled library. EasiGP already contains a fix for the known rpy2 multi-threading cause. Remaining causes are mixed library versions (recreate the environment), running out of memory, or many threads inside R. Set **R BLAS threads** to 1, reduce workers, and check the HPC error file for the printed stack trace. Completed tasks are kept, so a resubmitted job continues.
`TerminatedWorkerError` / `BrokenProcessPool` / `A worker process ... was unexpectedly terminated`	A parallel worker was killed, almost always for lack of memory. Reduce the number of workers or model-level workers, request more memory, or run a pilot job to measure the real need.
The R models are extremely slow during tuning	Keep **Use a cheaper MCMC length while searching** ticked (Tab 3), and consider **Fast tuning** with the exact search engine (Section 9.3).

Paths and operating-system details

What you see	Cause and what to do
R reports `cannot open file '...': No such file or directory` **for a file that exists, on Windows**	R on Windows has problems with non-English characters and very long paths. Keep EasiGP and your data in a short path with only English letters, digits and underscores, e.g. `C:\EasiGP`.
Files cannot be written, or are written slowly, in OneDrive/Dropbox folders	Synchronisation locks files while R or Python writes them. Keep EasiGP and its Result folder outside synchronised folders.
`Error in Sys.setlocale` / `During startup - Warning messages: Setting LC_CTYPE failed`	The system locale is missing (common in containers and on HPC). Harmless; to silence it set `LANG=C.UTF-8` before starting.

ASReml-R

What you see	Cause and what to do
`[ASReml] ASREML is enabled ... but the 'asreml' R package is not installed in this R environment`	ASReml-R was installed into a different R. Activate the EasiGP environment, start R from it and install the package there (Section 3.3). It is not available in the Docker images.
`[ASReml] ... does not appear to be licensed on this machine`	Activate the licence on the machine that runs the job (Tab 2 or `asreml::asreml.license.activate()`). Activation is per machine: a login node's licence does not cover compute nodes.
`undefined symbol: R_NilValue` **when loading asreml**	EasiGP makes R's symbols visible to ASReml automatically on Linux. If the error still appears, R_HOME points to an R without `lib/libR.so`; use the environment's R (auto-detect) and restart.
The asreml package refuses to install or load	The download must match the environment's R version (R `--version`) and operating system. Download the matching build from VSNi.
ASReml-R reports that it ran out of workspace	Raise **Workspace** and **Prediction workspace** in *Advanced ASReml-R settings* (e.g. 8gb).
`Data has N observations but the residual model implies M`	A spatial/temporal residual structure needs the complete trial layout. Keep **Include the entire dataset** on (it is locked on in this case) and check the row/column columns.
A task stops with a convergence failure	By default a non-converging fit stops the run. Simplify the model (fewer random terms or variance structures), or set *If this task fails to converge* to *Skip this task*.

18.5 Python and conda environment

What you see	Cause and what to do
`conda: command not found` / `'conda' is not recognized`	Use the **Anaconda Prompt** on Windows; on an HPC run `module load anaconda3` (or your cluster's equivalent); on macOS/Linux run `conda init` once and open a new terminal.
`ResolvePackageNotFound` / `PackagesNotFoundError` **when creating the environment**	The environment file does not match your operating system: use `environment_windows.yml` on Windows and `environment_linux.yml` on Linux. The files pin exact Linux or Windows builds, so on macOS use Docker (Route A) instead.
Creating the environment takes hours, or `LibMambaUnsatisfiableError`	Update conda first (`conda update -n base conda`; recent versions use the fast libmamba solver), make sure you are online, and try again. On an HPC, create it on the login node.
`CondaValueError: prefix already exists`	An environment called EasiGP already exists. Update it with `conda env update -f Environments/environment_<os>.yml --prune`, or remove it first: `conda env remove -n EasiGP`.
`streamlit: command not found` / `No module named 'streamlit'` / `No module named 'torch_geometric'`	The environment is not activated in this terminal: `conda activate EasiGP`.
`A module that was compiled using NumPy 1.x cannot be run in NumPy 2.x` / `numpy.core.multiarray failed to import`	A package was installed or upgraded with pip and replaced the pinned numpy. Avoid `pip install -U` inside the EasiGP environment; restore it with `conda env update ... --prune` or recreate it.
`Port 8501 is already in use`	Another EasiGP or Streamlit app is running. Close it, or start with `--server.port 8502` and open http://localhost:8502.
`torch.cuda.OutOfMemoryError: CUDA out of memory`	The GPU memory is too small for the GAT or MLP settings. Lower Batch size, Neuron numbers or Number of heads, or let the model run on the CPU.

18.6 HPC jobs and job scripts

What you see	Cause and what to do
`/bin/bash^M: bad interpreter: No such file or directory` / `$'\r': command not found`	The script has Windows line endings (it was generated or edited on Windows). Use **Convert an existing bash file's line endings** in Tab 7, or on the cluster run `dos2unix script.sh` or `sed -i 's/\r$//' script.sh`.
`CommandNotFoundError: Your shell has not been properly configured to use 'conda activate'` / `conda: command not found` **in the job output**	The job starts in a fresh shell. In **Additional setup lines** add `module load anaconda3`, `source $(conda info --base)/etc/profile.d/conda.sh` and `conda activate EasiGP`, and keep **Use login shell** ticked.
`ModuleNotFoundError` **or** `there is no package called` **only in batch jobs**	The job activated a different environment or R than your interactive session. Use exactly the same setup lines as you use interactively.
`Invalid account or account/partition combination specified` **(Slurm)** / `Unknown queue` **(PBS)**	Check **Account / project code** and **Partition / queue** with your cluster's documentation or `sacctmgr show assoc user=$USER`.
`QOSMaxSubmitJobPerUserLimit` / `Job violates accounting/QOS policy` / `qsub: Job exceeds queue resource limits`	The array or resource request is larger than allowed. Choose your **HPC cluster profile**, use fewer, larger batches (raise Batch size), or set **Max concurrently running array tasks**.
`DUE TO TIME LIMIT` / `walltime exceeded`	Increase the walltime or reduce the batch size, then resubmit the same job: finished tasks are kept.

What you see	Cause and what to do
`oom-kill event` / `Exceeded job memory limit` / `Killed`	Request more memory, reduce workers, or use the pilot job to measure memory. Resubmitting continues where it stopped.
Some batches never produced output	In Step 2, enter the expected number of batches and click **Check batch status**; resubmit only the listed batch IDs (e.g. `sbatch --array=5,17 script.sh`).
The GUI run stops when the SSH tunnel or interactive job ends	Option B runs live inside the GUI's job. For long analyses use Option A: the submitted job keeps running after you log out.
FLASH-P or the gene-location agent fails on compute nodes	Many compute nodes (e.g. on NCI Gadi) have no internet access. Run these steps on a node with internet, or supply the network and gene-location files yourself.

18.7 Frequently asked questions

How many individuals and markers do I need?

There is no hard minimum, but with fewer than about 100 individuals per task the test set is very small and accuracy estimates become noisy (hence the default minimum of 100). A few hundred to a few thousand individuals and a few thousand markers or more are typical. More markers rarely hurt accuracy but increase run time.

My accuracy is low. Is something wrong?

Not necessarily. Low-heritability traits, small training sets, and prediction across unrelated populations all give low accuracy. Check `Basic_stats.csv` for split sizes, compare with a simple model such as rrBLUP, and make sure the trait values are correct (e.g. no codes such as -9 used for missing data).

Why do I get slightly different results when I rerun?

The splits are reproducible, and most models are now seeded: rrBLUP, BayesB and RKHS seed their MCMC from the data, the tree models use a fixed seed, and the GAT models are seeded while **Fix the GAT random seed** is on. Small differences can remain for GBLUP, the MLP, GPU fits and runs with a different number of workers. Use several replicates and compare averages.

Can I stop a run and continue later?

Yes. Completed tasks are saved immediately. Start the same configuration again with the **same result folder name** and EasiGP continues with the unfinished tasks. Do not delete the run folder in between.

Which number should I report in a paper?

Usually the mean (or median) Pearson correlation on the test set over replicates, with its standard error, from `Metric_summary.xlsx`. State the split ratio, the number of replicates and the models used, keep the run's `*_config.json`, and cite EasiGP (below).

Can I predict individuals that have no phenotype at all?

EasiGP is designed to evaluate models by hiding known phenotypes. The test-set predictions in `Prediction_result_test.csv` show what such predictions look like; individuals without any phenotype are left out of the analysis.

Where do I get help?

Open an issue on the GitHub repository (https://github.com/ShunichiroT/EasiGP) with: what you did, the full error from the terminal or job files (Section 18.1), your operating system, whether you use Docker, Apptainer or conda, and for R problems the output of `python -m rpy2.situation`.

Appendix A. Glossary

Term	Meaning
Allele	One of the variants at a marker position. Genotype values count copies of one allele (0, 1 or 2).
BLUE / BLUP	Best linear unbiased estimate / prediction: an individual's trait value adjusted for field effects by a mixed model (Section 8.3).
Batch	In Parallel mode, a group of tasks processed by one HPC array sub-job.
bp / cM	Base pairs (physical position on the DNA) / centiMorgans (genetic map distance). Use one unit consistently in all position files.
Circos plot	A circular genome map showing marker effects, genes and interactions in rings and links (Sections 12, 16.7).
Consensus ring	A circos ring summarising where the models agree.
Data leakage	Accidentally using test-set information during training, which makes accuracy look too good. EasiGP prevents it.
Ensemble	A combination of the predictions (and effects) of the selected models.
Epistasis	When the effect of one gene depends on another gene. Interaction scores are a model-based signal of this.
GEBV	Genomic estimated breeding value: a predicted trait value from DNA markers.
Genotype	The table of marker values (0/1/2) for every individual.
GRM	Genomic relationship matrix: how genetically similar every pair of individuals is, computed from markers.
Heritability (h^2)	Share of trait variation due to genetics (0–1). Limits achievable accuracy.
HPC	High-performance computing cluster, where jobs are submitted through a scheduler (Slurm or PBS).
Hyperparameter	A setting that controls how a model learns (e.g. number of trees). Can be tuned automatically. Every one is explained in Appendix C.
Interaction	A score for how strongly a pair of markers acts together in a model.
LD (linkage disequilibrium)	Nearby markers inherited together, so they carry similar information.
MAF	Minor allele frequency: how common the rarer allele of a marker is.
Marker / SNP	A position in the genome where individuals differ; SNP = single-nucleotide polymorphism.
Marker effect	How strongly a marker contributes to a model's predictions (coefficients or Shapley values).
MSE	Mean squared error between predicted and measured values; lower is better.
Overfitting	When a model learns the training data too closely (including noise) and predicts new individuals poorly.
Pearson correlation (r)	Agreement between predicted and measured values (−1 to 1); the usual "prediction accuracy".
Phenotype	The measured trait values (e.g. yield, height).
Population	A group of related individuals (e.g. one family or breeding cross). The second column of the data files.

Term	Meaning
Replicate / iteration	A repeat of the random train/test split with a different random draw. More replicates show how stable the results are.
Shapley value / SHAP	A method that fairly shares a model’s prediction among the markers, used to compute marker effects for any model.
Split ratio	How individuals are divided into training, (validation) and test sets, e.g. 0.8 or (0.8, 0.1, 0.1).
Task (scenario)	One unit of work: a population (or population pair) × phenotype × split ratio × replicate. Every selected model is fitted once per task.
Training / test set	Individuals the model learns from / individuals used only to measure accuracy.
Validation set	Held-out individuals used to tune hyperparameters or learn ensemble weights. The test set is never used for this.

Appendix B. Configuration checklist

Step	Check
Data	Files found (green messages); first two columns are ID and population; target phenotype(s) selected; result folder named.
Split	Tuple ratio or validation set if tuning or weighted ensembles are used; enough replicates for your purpose.
Preprocessing	Only the sections you need are ticked; ASReml-R licensed if used.
Models	Models from different families ticked; effect and interaction settings reasonable for run time.
Ensemble	Naive and/or weighted methods selected.
Plots	Circos files set (same chromosome names and units); plot sections ticked.
Run	Option B for a test; Option A with setup lines, account and queue for HPC.
After the run	Check Basic_stats.csv (split sizes), Metric_summary.xlsx and the circos plots; keep the *_config.json with your results.

Appendix C. Hyperparameter reference

This appendix explains every setting in every model panel of Tab 3. You do not need to change any of them for a first analysis: the defaults are sensible starting points. Use this appendix when you want to understand a setting, tune a model by hand, or decide which settings automatic tuning should search.

How to read the tables

- **Model settings** change how the model learns and therefore its predictions. **Explanation settings** only change how marker effects and interactions are computed afterwards; they never change the predictions or the accuracy.
- **Default** is the value the GUI starts with. “on/off” refers to a tick box.
- **Tuning range** is the range automatic tuning searches (Section 9.3). “—” means the setting is never tuned. “(log)” means the range is searched on a logarithmic scale, so small values are explored as thoroughly as large ones. ★ marks the settings EasiGP treats as the most influential for that model: tune or adjust these first. With **Fast tuning** and **Search space = priority** (Section 9.3), only the ★ settings are searched.
- Settings marked *(shown when ...)* appear in the GUI only for that choice. Explanation settings appear only when the matching **Return ...?** box is ticked.

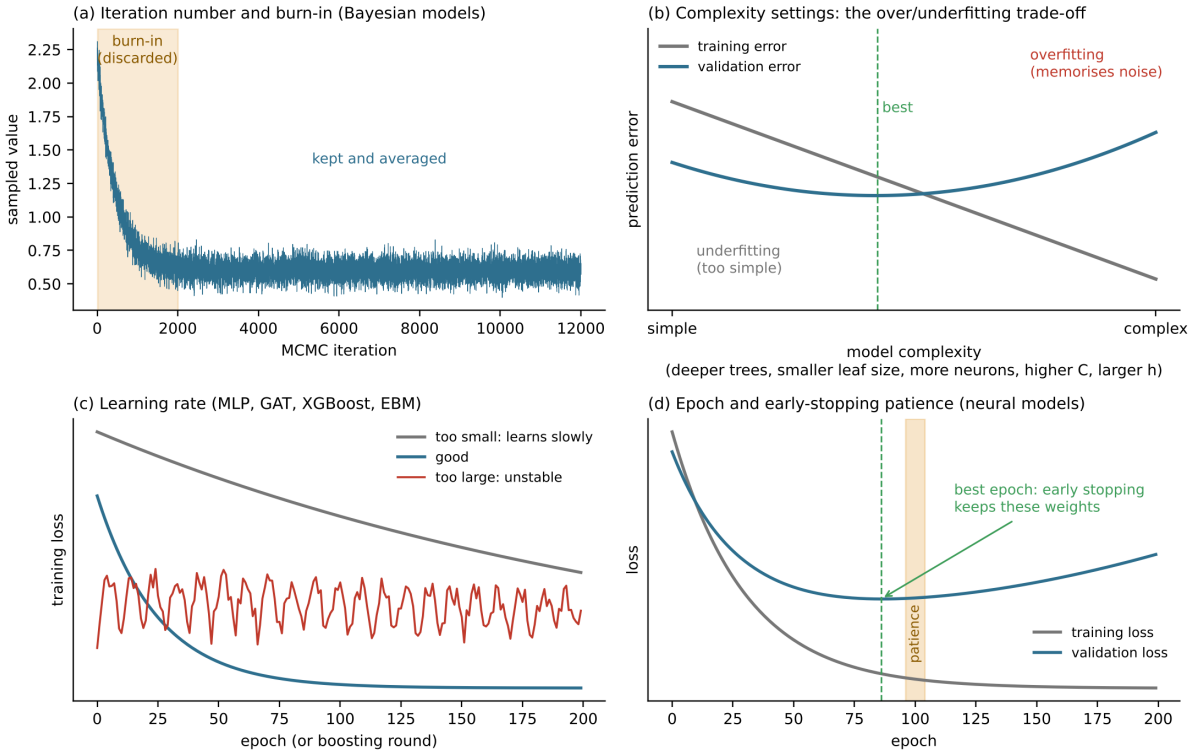


Figure 23. What the most common settings do (illustrative). (a) Bayesian models sample many times; the first rounds (burn-in) are discarded. (b) Complexity settings trade underfitting against overfitting; the best value is where the validation error is lowest. (c) The learning rate controls how fast neural and boosting models learn. (d) Too many epochs overfit; early stopping keeps the weights from the best epoch.

C.1 Which setting should I change?

What you see	Likely cause	Settings to try
Training accuracy much higher than test accuracy	Overfitting: the model memorises noise	Trees: raise *Minimum samples per leaf*, lower *Maximum tree depth*. XGBoost: lower *Learning rate*, raise *L2 regularisation*. SVR: lower *Constraint*. Neural models: raise *Dropout* or *Decay*, fewer *Epoch*, use early stopping.
Both training and test accuracy low	Underfitting: the model is too simple, or the trait has low heritability	Trees: allow deeper trees. Neural models: more *Neuron numbers* or *Epoch*, or a higher *Learning rate*. Also compare with rrBLUP: if all models are low, the trait itself is hard to predict.
Results change between identical runs	Too little sampling or training	BGLR models: raise *Iteration number* and *Burn-in*. RF/ExtraTrees: more trees. Neural models: lower the learning rate.
A model predicts almost the same value for everyone	Over-strong regularisation or a mis-scaled setting	SVR: keep *Scale epsilon and C by the trait SD* on and lower *Epsilon*. KNN: fewer neighbours. Neural models: check the learning rate.
The run is too slow	Explanation settings or large models	Lower *Max markers considered* and the *Number of samples* fields, keep interactions off where not needed, fewer trees or epochs, use **Fast tuning**.

C.2 Explanation methods in plain words

Several panels offer a choice of **Marker effect method** and **Marker interaction method**. They only change how the model is explained, never its predictions.

Method	Used by	What it does
closed_form	rrBLUP, BayesB, GBLUP	Converts the fitted model exactly into a Shapley value per marker, with no extra fitting. Fast and exact.
beta_direct	rrBLUP, BayesB	Reports the fitted marker effect (beta) itself.
iml_shapley	GBLUP, RKHS	Estimates Shapley values by refitting the model many times. Slow and approximate; kept for comparison.
shap (TreeSHAP)	RF, ExtraTrees, XGBoost	Exact Shapley values computed from the fitted trees.
feature_importance	RF, ExtraTrees, XGBoost	The model's built-in importance score. Very fast, but only a ranking, not on the same scale as Shapley values.
baseline_shap	RKHS, SVR, KNN	A marker's effect is the average change in prediction when its value is switched to that of real training individuals. Deterministic; uses only genotype combinations that exist in your data. For interactions, the same idea applied to pairs, on the same scale as tree interactions.
shap_predictor / kernel_shap	RKHS / SVR, KNN	The shap package's KernelExplainer: a sampling-based Shapley estimate.
deep_shap	MLP	The shap package's DeepExplainer for neural networks.
GAT marker effects	GAT models	A gradient-based attribution of each marker (or gene) in the trained network.
pairwise_shap	RF, ExtraTrees, XGBoost	Exact pairwise TreeSHAP interaction values.

Method	Used by	What it does
shapiq_tree	RF, ExtraTrees, XGBoost	Exact pairwise values like pairwise_shap, and can also search three-way (or higher) interactions.
nid	MLP	Neural Interaction Detection: reads likely interacting pairs from the trained weights. Almost free; an approximate ranking.
Native pairwise terms	EBM	The interaction terms the EBM itself learned while fitting.
Attention	GAT models	How strongly the network attends to each connection between markers (or genes). Written to Attention.csv.

C.3 rrBLUP, BayesB and GBLUP

These three models are fitted in R with BGLR, by MCMC sampling (Figure 23a). They are additive models: they do not return marker-pair interactions.

rrBLUP

Model setting	Default	Tuning range	What it does
Iteration number	12,000	2,000 – 20,000, step 2,000	How many MCMC sampling rounds the Bayesian fit runs. More rounds give a steadier estimate but take proportionally longer. Increase (e.g. 20,000–30,000) if results vary a lot between identical runs.
Burn-in	2,000	200 – 5,000, step 200	How many of the first rounds are thrown away while the sampler “warms up” from its arbitrary starting point. Must be smaller than the iteration number; about 10–20 % of it is typical.
Prior degrees of freedom (df0)	5	1 – 20, step 1	How firmly the model holds its prior belief about the size of marker effects. Higher = effects shrunk more strongly towards zero; lower = the data decide more. Rarely needs changing.
Expected proportion of variance explained (R2)	0.5	0.1 – 0.9 ★	Your rough guess of how much of the trait the markers explain overall; it sets how strongly effects are shrunk. Higher allows larger effects. A value near the trait’s heritability is a sensible choice.
Explanation setting	**Default**	**Tuning range**	**What it does**
Marker effect method	closed_form	—	closed_form (default): an exact Shapley value per marker computed from the fitted effects, on the same scale as other models’ Shapley effects. beta_direct: the fitted marker effect itself.

When to change: the defaults suit most data. Raise *Iteration number* (and burn-in) if repeated runs give noticeably different accuracies; set *R2* close to the trait’s heritability if you know it.

BayesB

Model setting	Default	Tuning range	What it does
Iteration number	12,000	2,000 – 20,000, step 2,000	How many MCMC sampling rounds the Bayesian fit runs. More rounds give a steadier estimate but take proportionally longer. Increase (e.g. 20,000–30,000) if results vary a lot between identical runs.
Burn-in	2,000	200 – 5,000, step 200	How many of the first rounds are thrown away while the sampler “warms up” from its arbitrary starting point. Must be smaller than the iteration number; about 10–20 % of it is typical.
Prior probability of a nonzero effect (probIn)	0.5	0.01 – 0.9 ★	BayesB’s key setting: the share of markers assumed to have a real effect. Small values (0.01–0.1) suit traits controlled by a few major genes; values near 0.5 or above behave more like rrBLUP.
Prior counts (counts)	10	2 – 100, step 2	How firmly the probIn guess above is held. Higher = the model sticks to probIn; lower = the data can move it more easily.
Explanation setting	**Default**	**Tuning range**	**What it does**
Marker effect method	closed_form	—	closed_form (default): an exact Shapley value per marker computed from the fitted effects. beta_direct: the fitted marker effect itself.

When to change: *probIn* is the setting that matters most (★). For traits with a few known major genes try 0.01–0.1; for highly polygenic traits keep 0.5 or tune it.

GBLUP

Model setting	Default	Tuning range	What it does
Iteration number	12,000	2,000 – 20,000, step 2,000	How many MCMC sampling rounds the Bayesian fit runs. More rounds give a steadier estimate but take proportionally longer. Increase (e.g. 20,000–30,000) if results vary a lot between identical runs.
Burn-in	2,000	200 – 5,000, step 200	How many of the first rounds are thrown away while the sampler “warms up” from its arbitrary starting point. Must be smaller than the iteration number; about 10–20 % of it is typical.
Explanation setting	**Default**	**Tuning range**	**What it does**
Return marker effect?	off	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of samples for Shapley scores	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.
Max markers considered for Shapley scores ("all" for every marker)	500	—	Only the top markers (by correlation with the trait) are scored; the rest are reported as 0. Lower = much faster.

Explanation setting	Default	Tuning range	What it does
Number of iterations for Shapley scores	200	—	MCMC rounds used in each refit of the iml_shapley route. Much smaller than the main iteration number because the refit is repeated many times.
Burn-in for Shapley scores	50	—	Warm-up rounds discarded in each iml_shapley refit; must be smaller than the value above.
Marker effect method	closed_form	—	closed_form (default): converts the predicted breeding values into exact marker effects; fast. iml_shapley: refits the model many times; much slower and approximate, kept for comparison.

When to change: only the sampling length can be tuned. Marker effects are off by default: tick *Return marker effect?* to show GBLUP on the circos plot (the default closed_form method is fast).

C.4 RKHS

Model setting	Default	Tuning range	What it does
Iteration number	12,000	2,000 – 20,000, step 2,000	How many MCMC sampling rounds the Bayesian fit runs. More rounds give a steadier estimate but take proportionally longer. Increase (e.g. 20,000–30,000) if results vary a lot between identical runs.
Burn-in	2,000	200 – 5,000, step 200	How many of the first rounds are thrown away while the sampler “warms up” from its arbitrary starting point. Must be smaller than the iteration number; about 10–20 % of it is typical.
Kernel bandwidth (h)	1.0	0.1 – 5.0 (log) ★	How quickly genetic similarity fades with genetic distance. Large h = only very similar individuals count as alike (local, flexible); small h = distant individuals still count (smooth, closer to GBLUP).

Explanation setting	Default	Tuning range	What it does
Return marker effect?	off	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of samples for Shapley scores	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.
Max markers considered for Shapley scores ("all" for every marker)	500	—	Only the top markers (by correlation with the trait) are scored; the rest are reported as 0. Lower = much faster.
Number of iterations for Shapley scores	200	—	MCMC rounds used in each refit of the iml_shapley route. Much smaller than the main iteration number because the refit is repeated many times. *(shown when Marker effect method = iml_shapley)*

Explanation setting	Default	Tuning range	What it does
Burn-in for Shapley scores	50	—	Warm-up rounds discarded in each iml_shapley refit; must be smaller than the value above. *(shown when Marker effect method = iml_shapley)*
Marker effect method	baseline_shap	—	baseline_shap (default): effect = average change in prediction when the marker is switched to the value of real training individuals; exact and deterministic. shap_predictor: KernelSHAP on an exact copy of the fit. iml_shapley: refits the model many times (slow).
Background sample size for marker effect	50	—	Number of real training individuals used as the reference when a marker is “switched” to estimate its effect. Larger = smoother, slower. *(shown when Marker effect method = baseline_shap)*
Return marker-pair interactions?	off	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Max markers considered for interaction search ("all" for every marker)	500	—	Only pairs among the top-ranked markers are checked. The number of pairs grows with the square of this value, so it is the main control of interaction run time.
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.
Number of individuals used to explain the model	50	—	How many training individuals the interaction scores are averaged over. More = more stable, slower.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Marker interaction method	baseline_shap	—	baseline_shap: scores pairs using only genotype combinations that really occur in the training population, on the same scale as RF’s TreeSHAP interactions.
Interaction pre-screen (speed vs. completeness)	off	—	off = score every candidate pair exactly. marginal_pd = rank pairs cheaply first and score only the best; much faster but can miss pairs whose markers have no effect on their own. Options: off, marginal_pd. *(shown when Marker interaction method = baseline_shap)*
Fraction of pairs scored exactly when pre-screening (%)	all	—	With the pre-screen on: the percentage of pairs that get an exact score. Lower = faster, but more pairs may be missed. *(shown when Marker interaction method = baseline_shap)*

When to change: the *Kernel bandwidth (h)* is the key setting (★); tuning it on a log scale is usually worthwhile. Small h behaves like GBLUP; large h captures more local, non-additive similarity but can overfit.

C.5 Tree models: RF, ExtraTrees and XGBoost

RF (Random Forest)

Model setting	Default	Tuning range	What it does
Tree number	100	100 – 2,000, step 100	How many decision trees are averaged. More trees = steadier predictions; gains usually level off well before 1,000.
Maximum features per tree	1.0	sqrt, log2, 0.5, 0.8, 1.0 ★	How many markers each split may choose from: sqrt or log2 = a small random subset (more varied trees, often better with many markers); 1.0 or None = all markers. You can also type a fraction such as 0.3.
Maximum samples per tree	None	None, 0.5, 0.7, 0.9	How many training individuals each tree is built from (drawn with replacement). None = as many as the training set; smaller fractions make trees more different from each other.
Maximum tree depth	None	None, 5, 10, 15, 25	How many levels of splits a tree may have. None = unlimited (can overfit noisy traits); smaller values give simpler, more general trees.
Minimum samples per leaf in each tree	1	1 – 20, step 1 ★	The smallest group of individuals a leaf may contain. 1 fits fine detail; 5–20 smooths predictions and reduces overfitting.
Explanation setting	**Default**	**Tuning range**	**What it does**
Return marker effect?	on	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of individuals used to explain the model (effect method "shap" or "shapiq" only)	30	—	How many test individuals are explained when computing SHAP marker effects. *(shown when Marker effect method = shap)*
Marker effect method	shap	—	shap (default): exact TreeSHAP values, comparable with other models’ Shapley effects. feature_importance: the forest’s built-in importance; fast but only a ranking.
Return marker effect for interactions?	on	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Number of samples for marker effect interactions	30	—	How many test individuals the interaction search explains. More = more representative, but slower.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Max markers considered for interaction search ("all" for every marker)	500	—	Only pairs among the top-ranked markers are checked. The number of pairs grows with the square of this value, so it is the main control of interaction run time.
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.

Explanation setting	Default	Tuning range	What it does
Marker interaction method	pairwise_shap	—	pairwise_shap (default): exact pairwise TreeSHAP interaction values. shapiq_tree: also exact for pairs and can additionally search three-way (or higher) interactions.
Interaction orders to search for (shapiq_tree method only, e.g. "2" or "2,3")	2	—	“2” = pairs only; “2,3” also looks for three-marker interactions (drawn as hub-and-spoke glyphs on the circos plot). Higher orders are much slower. *(shown when Marker interaction method = shapiq_tree)*
Shapley interaction index (shapiq_tree method only)	k-SII	—	Which mathematical definition of an interaction is used. Keep k-SII unless you want to compare definitions. Options: k-SII, FSII, STII. *(shown when Marker interaction method = shapiq_tree)*
Coalition sample budget for 3-way-or-higher shapiq_tree search (shapiq_tree method only)	4,096	—	Sampling budget for three-way (or higher) searches. Higher = more accurate, slower. No effect on pair-only searches. *(shown when Marker interaction method = shapiq_tree)*

When to change: *Maximum features per tree* and *Minimum samples per leaf* (★) matter most. For noisy traits raise the leaf size to 5–10. More trees rarely lower accuracy; they only cost time. Interactions are on by default for RF only.

ExtraTrees

Model setting	Default	Tuning range	What it does
Tree number	100	100 – 2,000, step 100	How many decision trees are averaged. More trees = steadier predictions; gains usually level off well before 1,000.
Maximum features per tree	1.0	sqrt, log2, 0.5, 0.8, 1.0 ★	How many markers each split may choose from: sqrt or log2 = a small random subset (more varied trees, often better with many markers); 1.0 or None = all markers. You can also type a fraction such as 0.3.
Bootstrap sample fraction ('None' = ExtraTrees' own default: no resampling)	None	None, 0.5, 0.7, 0.9	None = every tree sees the whole training set (the classic ExtraTrees). A fraction switches on RF-style resampling.
Maximum tree depth	None	None, 5, 10, 15, 25	How many levels of splits a tree may have. None = unlimited (can overfit noisy traits); smaller values give simpler, more general trees.
Minimum samples per leaf in each tree	1	1 – 20, step 1 ★	The smallest group of individuals a leaf may contain. 1 fits fine detail; 5–20 smooths predictions and reduces overfitting.

Explanation setting	Default	Tuning range	What it does
Return marker effect?	on	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.

Explanation setting	Default	Tuning range	What it does
Number of individuals used to explain the model (effect method "shap" or "shapiq" only)	30	—	How many test individuals are explained when computing SHAP marker effects. *(shown when Marker effect method = shap)*
Marker effect method	shap	—	shap (default): exact TreeSHAP values. feature_importance: built-in importance; fast but only a ranking.
Return marker effect for interactions?	off	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Number of samples for marker effect interactions	30	—	How many test individuals the interaction search explains. More = more representative, but slower.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Max markers considered for interaction search ("all" for every marker)	500	—	Only pairs among the top-ranked markers are checked. The number of pairs grows with the square of this value, so it is the main control of interaction run time.
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.
Marker interaction method	pairwise_shap	—	pairwise_shap (default): exact pairwise TreeSHAP values. shapiq_tree: can also search three-way (or higher) interactions.
Interaction orders to search for (shapiq_tree method only, e.g. "2" or "2,3")	2	—	“2” = pairs only; “2,3” also looks for three-marker interactions (drawn as hub-and-spoke glyphs on the circos plot). Higher orders are much slower. *(shown when Marker interaction method = shapiq_tree)*
Shapley interaction index (shapiq_tree method only)	k-SII	—	Which mathematical definition of an interaction is used. Keep k-SII unless you want to compare definitions. Options: k-SII, FSII, STII. *(shown when Marker interaction method = shapiq_tree)*
Coalition sample budget for 3-way-or-higher shapiq_tree search (shapiq_tree method only)	4,096	—	Sampling budget for three-way (or higher) searches. Higher = more accurate, slower. No effect on pair-only searches. *(shown when Marker interaction method = shapiq_tree)*

When to change: as for RF. Keep the bootstrap fraction at None to get the classic ExtraTrees behaviour that makes it a useful alternative to RF in an ensemble.

XGBoost

Model setting	Default	Tuning range	What it does
Tree number (n_estimators)	100	50 – 500, step 50	How many boosting rounds (trees) are added one after another, each correcting the previous ones. Tune together with the learning rate.

Model setting	Default	Tuning range	What it does
Maximum tree depth	6	3, 4, 6, 8, 10 ★	How deep each boosted tree may grow (default 6). Deeper trees capture more interactions but overfit more easily.
Learning rate	0.1	0.01 – 0.3 (log) ★	How much each new tree corrects the previous ones. Smaller values need more trees but often generalise better.
Row subsample fraction	1.0	0.5 – 1.0	Share of training individuals randomly used for each tree. Values below 1.0 add randomness and can reduce overfitting.
Column subsample fraction per tree	1.0	0.3 – 1.0	Share of markers randomly available to each tree. Values below 1.0 add randomness, useful with many markers.
L2 regularisation (reg_lambda)	1.0	0.0 – 5.0 ★	A penalty on large leaf values; higher = more cautious, smoother model.
Explanation setting	**Default**	**Tuning range**	**What it does**
Return marker effect?	on	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of individuals used to explain the model (effect method "shap" or "shapiq" only)	30	—	How many test individuals are explained when computing SHAP marker effects. *(shown when Marker effect method = shap)*
Marker effect method	shap	—	shap (default): exact TreeSHAP values. feature_importance: built-in gain importance; fast but only a ranking.
Return marker effect for interactions?	off	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Number of samples for marker effect interactions	30	—	How many test individuals the interaction search explains. More = more representative, but slower.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Max markers considered for interaction search ("all" for every marker)	500	—	Only pairs among the top-ranked markers are checked. The number of pairs grows with the square of this value, so it is the main control of interaction run time.
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.
Marker interaction method	pairwise_shap	—	pairwise_shap (default): exact pairwise TreeSHAP values. shapiq_tree: can also search three-way (or higher) interactions.
Interaction orders to search for (shapiq_tree method only, e.g. "2" or "2,3")	2	—	“2” = pairs only; “2,3” also looks for three-marker interactions (drawn as hub-and-spoke glyphs on the circos plot). Higher orders are much slower. *(shown when Marker interaction method = shapiq_tree)*

Explanation setting	Default	Tuning range	What it does
Shapley interaction index (shapiq_tree method only)	k-SII	—	Which mathematical definition of an interaction is used. Keep k-SII unless you want to compare definitions. Options: k-SII, FSII, STII. *(shown when Marker interaction method = shapiq_tree)*
Coalition sample budget for 3-way-or-higher shapiq_tree search (shapiq_tree method only)	4,096	—	Sampling budget for three-way (or higher) searches. Higher = more accurate, slower. No effect on pair-only searches. *(shown when Marker interaction method = shapiq_tree)*

When to change: *Learning rate*, *Maximum tree depth* and *L2 regularisation* (★). A common recipe is a lower learning rate (e.g. 0.05) with more trees (300+). Shallow trees (3–4) suit mostly additive traits.

C.6 EBM

Model setting	Default	Tuning range	What it does
Learning rate	0.04	0.01 – 0.2 (log)	How much each boosting round corrects the previous ones. Kept small on purpose (0.04) so the learned curves stay smooth.
Maximum leaves per tree	3	2 – 6, step 1	How many leaves each small boosting tree may have. EBMs use very shallow trees by design.
Minimum samples per leaf	4	2 – 20, step 2	The smallest group of individuals a leaf may contain. Larger values give smoother curves.
Outer bags	8	2 – 16, step 2	How many copies of the model are trained on resampled data and averaged. More = smoother, more stable, slower.
Search for marker-pair interactions?	off	—	Lets the EBM fit pairwise terms during training. Unlike other models this changes the model itself, not only what is reported.
Number of interaction terms to search for ("all" = this model's own default)	10	—	How many marker pairs the EBM adds as pairwise terms while fitting (a count of terms, not a marker shortlist).
Explanation setting	**Default**	**Tuning range**	**What it does**
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest of the pairwise terms the EBM found while fitting.

When to change: the defaults are deliberately conservative. Ticking *Search for marker-pair interactions?* changes the model itself and can improve accuracy for traits with gene-by-gene effects, at extra cost.

C.7 SVR

Model setting	Default	Tuning range	What it does
Kernel type	rbf	linear, poly, rbf, sigmoid ★	The similarity function used to compare individuals. rbf (default) captures curved, non-linear patterns; linear assumes additive effects; poly and sigmoid are alternatives to try if rbf underperforms. Options: linear, poly, rbf, sigmoid, precomputed.

Model setting	Default	Tuning range	What it does
Epsilon	0.5	0.001 – 2.0 (log) ★	A tolerance band: prediction errors smaller than epsilon are ignored. Larger = simpler, less sensitive model; smaller = fits the data more closely.
Constraint	1.0	0.01 – 100.0 (log) ★	The usual “C”: how hard the model tries to fit the training data. Higher = closer fit (risk of overfitting); lower = smoother, more general model.
Scale epsilon and C by the trait SD	on	—	Expresses Epsilon and Constraint relative to the trait’s spread, so the same values work for traits in any unit. Keep on; turning it off can make SVR predict one constant value.
Dimension for poly kernel	3	2 – 5, step 1	Only for the poly kernel: the polynomial degree. Higher = more complex curves, more overfitting risk.
Gamma	scale	scale, auto	How far the influence of one individual reaches (rbf, poly, sigmoid). scale (default) picks a sensible value from your data; you can also type a number. Higher = more local.
Independent term (coef0)	0.0	-1.0 – 1.0	Only for poly and sigmoid kernels: shifts the kernel by a constant. Usually left at 0.
Explanation setting	**Default**	**Tuning range**	**What it does**
Return marker effect?	off	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of samples for Shapley scores	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.
Max markers considered for Shapley scores ("all" for every marker)	500	—	Only the top markers (by correlation with the trait) are scored; the rest are reported as 0. Lower = much faster.
Background sample size for Shapley scores	50	—	Number of reference individuals used as the “typical” baseline for marker effects. 50 is usually enough.
Number of coalition samples for Shapley scores (nsamples)	200	—	Only for kernel_shap: how many random marker combinations are tried. Higher = more accurate, slower.
Marker effect method	baseline_shap	—	baseline_shap (default): average change in prediction when the marker is switched to real training values; deterministic. kernel_shap: the shap package’s KernelExplainer (sampling-based).
Return marker-pair interactions?	off	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Max markers considered for interaction search ("all" for every marker)	500	—	Only pairs among the top-ranked markers are checked. The number of pairs grows with the square of this value, so it is the main control of interaction run time.

Explanation setting	Default	Tuning range	What it does
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.
Background sample size for interaction search	100	—	How many test individuals are used to average out all other markers when scoring a pair. Larger = more stable, slower.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Marker interaction method	baseline_shap	—	baseline_shap: scores pairs using only genotype combinations that really occur in the training population (a model-agnostic estimate).
Interaction pre-screen (speed vs. completeness)	off	—	off = score every candidate pair exactly. marginal_pd = rank pairs cheaply first and score only the best; much faster but can miss pairs whose markers have no effect on their own. Options: off, marginal_pd. *(shown when Marker interaction method = baseline_shap)*
Fraction of pairs scored exactly when pre-screening (%)	all	—	With the pre-screen on: the percentage of pairs that get an exact score. Lower = faster, but more pairs may be missed. *(shown when Marker interaction method = baseline_shap)*

When to change: *Kernel type*, *Epsilon* and *Constraint* (★). Keep *Scale epsilon and C by the trait SD* ticked. If SVR predicts nearly the same value for everyone, lower Epsilon.

C.8 KNN

Model setting	Default	Tuning range	What it does
Number of neighbours	5	1 – 30, step 1 ★	k: how many of the most genetically similar training individuals are averaged. Small k = local detail but noisy; large k = smooth but can blur real differences.
Neighbour weighting	uniform	uniform, distance ★	uniform = all k neighbours count equally; distance = closer neighbours count more. Options: uniform, distance.
Distance metric power (p)	2	1 – 3, step 1	How genetic distance is measured: 1 = Manhattan (sum of differences), 2 = Euclidean (default). Higher values emphasise the single largest marker difference.

Explanation setting	Default	Tuning range	What it does
Return marker effect?	off	—	Computes each marker’s contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of samples for Shapley scores	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.
Max markers considered for Shapley scores ("all" for every marker)	500	—	Only the top markers (by correlation with the trait) are scored; the rest are reported as 0. Lower = much faster.

Explanation setting	Default	Tuning range	What it does
Background sample size for Shapley scores	50	—	Number of reference individuals used as the “typical” baseline for marker effects. 50 is usually enough.
Number of coalition samples for Shapley scores (nsamples)	200	—	Only for kernel_shap: how many random marker combinations are tried. Higher = more accurate, slower.
Marker effect method	baseline_shap	—	baseline_shap (default): exact Shapley values against real training individuals, with a fast KNN-specific algorithm. kernel_shap: sampling-based KernelExplainer.
Return marker-pair interactions?	off	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Max markers considered for interaction search ("all" for every marker)	500	—	Only pairs among the top-ranked markers are checked. The number of pairs grows with the square of this value, so it is the main control of interaction run time.
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.
Background sample size for interaction search	100	—	How many test individuals are used to average out all other markers when scoring a pair. Larger = more stable, slower.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Marker interaction method	baseline_shap	—	baseline_shap: scores pairs using only genotype combinations that really occur in the training population (a model-agnostic estimate).
Interaction pre-screen (speed vs. completeness)	off	—	off = score every candidate pair exactly. marginal_pd = rank pairs cheaply first and score only the best; much faster but can miss pairs whose markers have no effect on their own. Options: off, marginal_pd. *(shown when Marker interaction method = baseline_shap)*
Fraction of pairs scored exactly when pre-screening (%)	all	—	With the pre-screen on: the percentage of pairs that get an exact score. Lower = faster, but more pairs may be missed. *(shown when Marker interaction method = baseline_shap)*

When to change: *Number of neighbours* and *Neighbour weighting* (★). Values of 10–30 neighbours with distance weighting are often more stable than the default of 5 for genomic data.

C.9 MLP (neural network)

Model setting	Default	Tuning range	What it does
Neuron numbers	30	8 – 256, step 8 ★	Size of the (first) hidden layer. More neurons can represent more complex patterns but need more data and overfit more easily.

Model setting	Default	Tuning range	What it does
Dropout	0	0.0 – 0.6 ★	Share of connections randomly switched off in each training step, to prevent overfitting. 0 = off; 0.1–0.5 if the model overfits.
Learning rate	0.0001	0.0001 – 0.1 (log) ★	Size of each weight update. Too high = unstable training; too low = very slow learning. The default (0.0001) is small; try 0.001 if learning is slow.
Decay	0.0005	1e-06 – 0.01 (log)	Weight decay: a small penalty on large weights that discourages overfitting. Higher = stronger regularisation.
Epoch	200	10 – 200, step 10	How many times the network sees the whole training set. Too few = underfitting; too many = memorising noise (watch the validation score).
Batch size	8	4 – 64, step 4	How many individuals are processed before each weight update. Small = more frequent, noisier updates; large = faster epochs, fewer updates.
Second hidden layer size	None	None, 16, 32, 64	Adds a second hidden layer of this size. None = one hidden layer. Deeper networks can capture more interactions but train slower and overfit small datasets.
Explanation setting	**Default**	**Tuning range**	**What it does**
Number of samples for Shapley scores	30	—	How many test individuals DeepSHAP explains (also used as its background set). More = more representative, slower.
Return marker-pair interactions?	off	—	Searches for pairs of markers that act together (circos links, Interaction.csv). Slower; the fields below apply only when ticked.
Max markers considered for interaction search ("all" for every marker)	500	—	Limits the interaction table to pairs among the top markers by effect. This only limits the size of the output, not the run time.
Output only the top N% of interactions ("all" for everything)	all	—	Keeps only the strongest pairs (e.g. 1 = top 1 %). “all” can produce a very large file.
Marker effect method	deep_shap	—	deep_shap: the shap DeepExplainer applied to the trained network. Always computed for every marker.
Marker interaction method	nid	—	nid: Neural Interaction Detection, read from the trained weights; almost free, but an approximate ranking.
Use marker effects to select markers for interaction search?	on	—	Re-uses the marker-effect ranking to choose which markers enter the interaction search, instead of a separate ranking. Leave on.

When to change: neural networks are the most sensitive to their settings, so tuning (★ settings: neurons, dropout, learning rate) is recommended. If training is slow to improve, raise the learning rate from its small default; if the model overfits, add dropout (0.1–0.3) or decay.

C.10 Graph attention (GAT) models

All four GAT models share the same training settings. They differ only in which connections the graph contains (Section 9.2) and in a few extra settings listed below.

GAT_infinitesimal and GAT_fully_connected

Model setting	Default	Tuning range	What it does
Neuron numbers	20	8 – 128, step 8 ★	Size of the (first) hidden layer. More neurons can represent more complex patterns but need more data and overfit more easily.
Dropout	0	0.0 – 0.6 ★	Share of connections randomly switched off in each training step, to prevent overfitting. 0 = off; 0.1–0.5 if the model overfits.
Learning rate	0.01	0.0001 – 0.1 (log) ★	Size of each weight update. Too high = unstable training; too low = slow learning.
Decay	0.0005	1e-06 – 0.01 (log) ★	Weight decay: a small penalty on large weights that discourages overfitting. Higher = stronger regularisation.
Epoch	40	10 – 200, step 10	How many times the network sees the whole training set. Too few = underfitting; too many = memorising noise (watch the validation score).
Batch size	8	4 – 64, step 4	How many individuals are processed before each weight update. Small = more frequent, noisier updates; large = faster epochs, fewer updates.
Number of heads	1	1 – 8, step 1	How many independent attention patterns are learned in parallel; each head can focus on a different kind of relationship. More heads = bigger, slower model.

Explanation setting	Default	Tuning range	What it does
Return marker effect?	on	—	Computes each marker's contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of samples for Shapley scores	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.

GAT_prior_knowledge

Same training settings as above, plus the RF edge selection and the interaction output:

Model setting	Default	Tuning range	What it does
Neuron numbers	20	8 – 128, step 8 ★	Size of the (first) hidden layer. More neurons can represent more complex patterns but need more data and overfit more easily.
Dropout	0	0.0 – 0.6 ★	Share of connections randomly switched off in each training step, to prevent overfitting. 0 = off; 0.1–0.5 if the model overfits.
Learning rate	0.01	0.0001 – 0.1 (log) ★	Size of each weight update. Too high = unstable training; too low = slow learning.

Model setting	Default	Tuning range	What it does
Decay	0.0005	1e-06 – 0.01 (log) ★	Weight decay: a small penalty on large weights that discourages overfitting. Higher = stronger regularisation.
Epoch	40	10 – 200, step 10	How many times the network sees the whole training set. Too few = underfitting; too many = memorising noise (watch the validation score).
Batch size	8	4 – 64, step 4	How many individuals are processed before each weight update. Small = more frequent, noisier updates; large = faster epochs, fewer updates.
Number of heads	1	1 – 8, step 1	How many independent attention patterns are learned in parallel; each head can focus on a different kind of relationship. More heads = bigger, slower model.
Selection rate for edges from RF (e.g. 10 = select the top 10% of the most important edges)	10	—	A Random Forest first finds marker pairs that seem to interact; only the top X % of pairs become graph connections. Lower = fewer, stronger connections and a faster model.
Explanation setting	**Default**	**Tuning range**	**What it does**
Return marker effects?	on	—	Computes each marker's contribution, used for the circos rings and Marker_effect.csv. Untick to save time if you only need accuracies.
Number of samples for marker effects	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.
Return marker-pair interactions?	off	—	Reports the interaction strengths the model already computed (with RF) to build its graph. Does not change training or predictions.

GAT_biological_prior_knowledge

Same training settings as above, plus the marker-info file and early stopping. The network, gene-location file, coordinate unit and mediated-edge settings are set on Tab 2 (Section 8.4) and are not shown in this panel:

Model setting	Default	Tuning range	What it does
Neuron numbers	20	8 – 128, step 8 ★	Size of the (first) hidden layer. More neurons can represent more complex patterns but need more data and overfit more easily.
Dropout	0	0.0 – 0.6 ★	Share of connections randomly switched off in each training step, to prevent overfitting. 0 = off; 0.1–0.5 if the model overfits.
Learning rate	0.01	0.0001 – 0.1 (log) ★	Size of each weight update. Too high = unstable training; too low = slow learning.
Decay	0.0005	1e-06 – 0.01 (log) ★	Weight decay: a small penalty on large weights that discourages overfitting. Higher = stronger regularisation.

Model setting	Default	Tuning range	What it does
Epoch	40	10 – 200, step 10	How many times the network sees the whole training set. Too few = underfitting; too many = memorising noise (watch the validation score).
Batch size	8	4 – 64, step 4	How many individuals are processed before each weight update. Small = more frequent, noisier updates; large = faster epochs, fewer updates.
Number of heads	1	1 – 8, step 1	How many independent attention patterns are learned in parallel; each head can focus on a different kind of relationship. More heads = bigger, slower model.
Marker info CSV path (chromosome, name, start, end)	(set on Tab 2)	—	Marker positions, used to place each marker inside a gene. You can reuse the circos marker_info.csv.
Early stopping patience (epochs, 0 = off)	0	—	Stops training when the validation error has not improved for this many epochs and keeps the best weights. 0 = off. Needs a validation split; 3–10 is typical.

Explanation setting	Default	Tuning range	What it does
Return marker effects?	on	—	Computes each gene's effect and copies it onto every marker inside that gene; markers outside all genes are reported as 0.
Number of samples for marker effects	30	—	How many test individuals are explained when computing marker effects. More = more representative, but slower.

When to change: *Neuron numbers*, *Dropout*, *Learning rate* and *Decay* (★) matter most. Start with few heads (1–2). For GAT_fully_connected, cost grows quickly with the number of markers, so reduce markers first (Tab 2). For GAT_prior_knowledge, a lower edge selection rate makes the model faster and more focused. For the biological-prior model, early stopping (patience 3–10) saves time when a validation split is used.

C.11 Weight-optimisation ensemble methods (Tab 4)

Each ticked method on Tab 4 has its own panel. All of them learn the weights on the validation set, so they need a (train, validation, test) split.

Nelder Mead

Setting	Default	What it does
Initial value	0.5	The starting weight of every model. Rarely matters; it only has to lie within the boundaries.
Minimum boundary	0.0	The smallest weight any model may get. 0 lets the search drop a weak model completely.
Maximum boundary	1	The largest weight any model may get.
fatol	1e-08	Stop the search when the fit improves by less than this. Smaller = more thorough, slower.
xatol	1e-08	Stop the search when the weights change by less than this between attempts.
Adaptive	on	Adjusts the search's step sizes to the number of models. Recommended with more than 4–5 models.

Setting	Default	What it does
Diversity penalty (lambda)	0.0	Optional pull of the weights towards equal weighting. 0 = off. Mainly useful with the dpt_ratio objective.
Objective	ensemble_mse	What is minimised on the validation set: ensemble_mse (default, the ensemble's own error; gives a genuine blend) or dpt_ratio (the Diversity Prediction Theorem ratio; tends to pick a single model).
Naive-shrinkage alpha	1.0	Blends the result towards the naive (equal-weight) ensemble: 1 = use the weights as found, 0 = exactly the naive ensemble. A safety margin against overfitting the validation set.
Diversity penalty method	ridge_to_uniform	How the diversity penalty is measured: ridge_to_uniform (default, distance from equal weights) or entropy (a softer penalty).
Vectorised objective	on	Computes the objective as one fast matrix operation. Same formula; leave on.

Bayesian optimisation

Setting	Default	What it does
Minimum boundary	0.0	The smallest weight any model may get. 0 lets the search drop a weak model completely.
Maximum boundary	10.0	The largest weight any model may get.
Iterations	50	How many weight combinations the Bayesian search tries.
Point numbers	3	Random combinations tried before the search starts learning (equal weighting is always tried first).
Allow duplicate points	on	Lets the search revisit a combination, so it never gets stuck. Leave on.
Diversity penalty (lambda)	0.0	Optional pull of the weights towards equal weighting. 0 = off. Mainly useful with the dpt_ratio objective.
Objective	ensemble_mse	What is minimised on the validation set: ensemble_mse (default, the ensemble's own error; gives a genuine blend) or dpt_ratio (the Diversity Prediction Theorem ratio; tends to pick a single model).
Naive-shrinkage alpha	1.0	Blends the result towards the naive (equal-weight) ensemble: 1 = use the weights as found, 0 = exactly the naive ensemble. A safety margin against overfitting the validation set.
Diversity penalty method	ridge_to_uniform	How the diversity penalty is measured: ridge_to_uniform (default, distance from equal weights) or entropy (a softer penalty).
Restarts	1	Independent searches with different random seeds; the best is kept. Each uses its own CPU core when available.
Batch suggestion	on	Suggests several candidates per round. Modest speed gain; safe to leave on.
Batch liar strategy	max	How not-yet-scored candidates in a batch are provisionally scored (max, mean, believer). Keep max.
Vectorised objective	on	Computes the objective as one fast matrix operation. Same formula; leave on.

Linear transformation

Setting	Default	What it does
Learning rate	0.005	Size of each weight update while the small network learns the weights. Larger = faster but can overshoot; smaller = steadier.
Epochs	150	How many passes over the validation set the weight-learning network makes. More can find better weights but risks overfitting the validation set.
Decay	0.01	A small penalty that discourages very large weights, so the ensemble does not over-rely on one model.
Batch size	2	How many validation individuals are used per weight update.
Patience value	10	Stops training once the fit has not improved for this many epochs in a row.
Number of MLP models	30	How many times the weight-learning network is trained from different random starts; the best run is kept.
Diversity penalty (lambda)	0.0	Optional pull of the weights towards equal weighting. 0 = off. Mainly useful with the dpt_ratio objective.
Use simplex (non-negative, sum-to-1) weights	off	Off: weights are unconstrained and can be negative. On: weights are always non-negative and sum to 1 (a pure blend).
Naive-shrinkage alpha	1.0	Blends the result towards the naive (equal-weight) ensemble: 1 = use the weights as found, 0 = exactly the naive ensemble. A safety margin against overfitting the validation set.

Analytic least-squares

Setting	Default	What it does
Ridge regularisation	1e-06	A tiny stabilising term for the exact solution when the models' predictions are highly correlated. Raise only if the log reports an unstable solve.
Naive-shrinkage alpha	1.0	Blends the result towards the naive (equal-weight) ensemble: 1 = use the weights as found, 0 = exactly the naive ensemble. A safety margin against overfitting the validation set.

When to change: the defaults are safe. If weighted ensembles do worse than the naive ensemble on the test set, lower *Naive-shrinkage alpha* (e.g. 0.5) or switch to Analytic least-squares, which has nothing to tune.

Citation. If you use EasiGP, please cite: Tomura S. et al. Ensemble AnalySis with Interpretable Genomic Prediction (EasiGP): computational tool for interpreting ensembles of genomic prediction models. *The Plant Genome*. doi:10.1002/tpg2.70138.